

Programs-of-Layers in LLMs through the Lens of Cortical Areas

Justus Westerhoff*

Berliner Hochschule für Technik (BHT), Berlin, Germany

justus.westerhoff@bht-berlin.de

Stephan Olbrich*

Berliner Hochschule für Technik (BHT), Berlin, Germany

stephan.olbrich@bht-berlin.de

Hatem Oraby

Humboldt-Universität zu Berlin, Berlin, Germany

hatem.oraby@hu-berlin.de

Matthew Evan Larkum

Humboldt-Universität zu Berlin, Berlin, Germany

matthew.larkum@hu-berlin.de

Felix Alexander Gers

Berliner Hochschule für Technik (BHT), Berlin, Germany

felixalexander.gers@bht-berlin.de

Abstract

Inference in large language models (LLMs) is conventionally a fixed-depth, fixed-order forward pass through every layer, regardless of how difficult the input is. The human brain does not work this way: using the thalamus as a central hub, it routes information flexibly to all regions of the cortex according to demand. Li et al. (2026) recently showed, with a system they call program-of-layers (POLAR), that transformers can be given an analogous flexibility if their layers are treated as a library of functions rather than a fixed sequence. Performance improves over the standard forward pass when each input is dynamically routed through an adaptive sequence of skipped or repeated contiguous layer blocks. We reconstructed POLAR’s diagnostic Monte Carlo Tree Search (MCTS) in more detail than the original paper and applied it across 5 models. We reproduced several of POLAR’s findings: skipping outperformed the standard pass, repeating outperformed skipping, and combining both outperformed either alone. Shorter programs sufficed for easier questions, while harder questions required more layer repeats. However, we failed to replicate the main claim regarding their learned router for single-shot inference: its top-ranked prediction consistently collapsed back to the standard pass, even though its top- k predicted programs, taken together, did show a real accuracy gain. Beyond reproduction, we find that a small number of generic programs are enough to solve most of the questions. We also provide a much deeper analysis of these programs’ structure and robustness: for example, we found that programs that correct errors are highly brittle: undoing even a single edit inside a program typically breaks the correction. Connecting this to the brain’s routing mechanisms, POLAR mirrors principles of thalamo-cortical coordination between cortical-area-like transformer layers. We publicly release the code at <https://datexis.github.io/RE-PoLar/>.

1 Introduction

Most large language models (LLMs) apply one fixed-depth architecture (Vaswani et al., 2017) to every input, regardless of how difficult it actually is. A conventional computer program, by contrast, is adaptive: program structure and complexity scale with the problem (e.g., through if-clauses and for-/while-loops). Similarly, the

*Shared first authorship; order chosen at random.

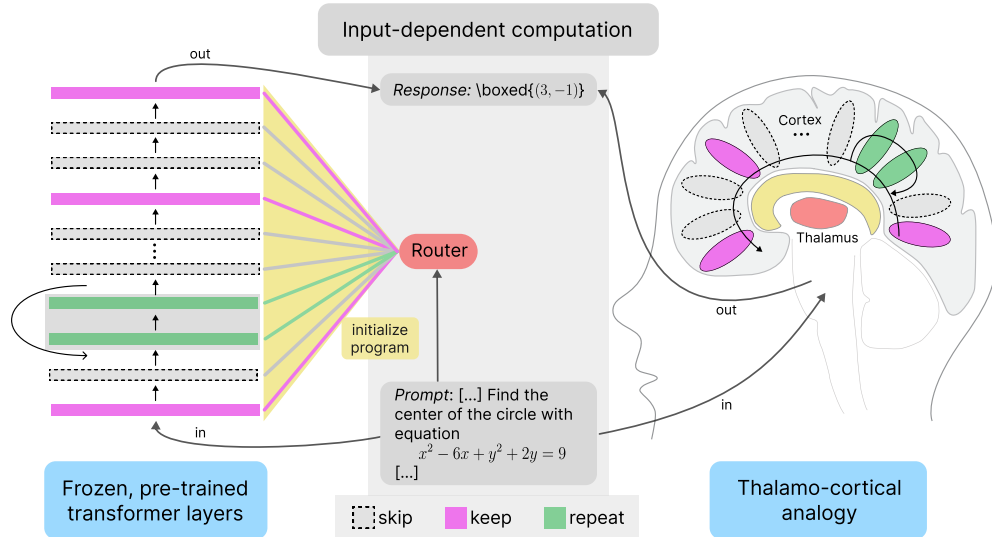


Figure 1: A program-of-layers (POLAR), predicted by a lightweight router, executes a schedule of *skip*, *keep*, and *repeat* operations over frozen, pretrained transformer layers (left). We read this coordinating role as *functionally* analogous to the thalamus routing computation across cortical areas (right). A transformer’s layer order is fixed only by training, not by necessity: a model *trained* to run its layers in order can nonetheless be usefully *executed* out of that order at inference time, a flexibility the brain already has by default.

human brain, as a resource-constrained system, follows an adaptive principle, allocating different amounts and types of neural computation to different tasks. This is reflected both in which brain circuits are recruited and in how long activity is sustained within them. Difficult tasks can engage recurrent processing: activity reverberates within and between cortical circuits until sufficient evidence has accumulated (Wang, 2002; Kar et al., 2019). Familiar tasks instead rely on faster, more automatic pathways (Poldrack et al., 2005). While anatomy constrains where sensory information first arrives, it does not fix how that information is routed afterward: feedback loops within the cortex, and between the cortex and the thalamus, can amplify some pathways and suppress others depending on context and the brain’s current behavioral state (Munn et al., 2026).

One axis of adaptivity LLMs do have at inference time is sequence length: a model can “think longer” by emitting more tokens (Wei et al., 2022), but the computation performed by any single forward pass through the network stays fixed regardless of how hard that step actually is. This raises a basic question for transformer inference: **can a fixed forward pass be adapted to recover more of a model’s latent capability, without retraining it?**

One family of answers changes the architecture and trains the model itself to be more recursive. Li et al. (2026) instead answer it without retraining the base model at all, treating the D pretrained layers $f_0, \dots, f_{D-1}$ of an LLM as a library of functions rather than a fixed pipeline. A per-input program-of-layers (POLAR) $\pi = (i_1, \dots, i_K)$, where each i_j is a layer index, selects a schedule of contiguous layer segments. Each segment is skipped, kept, or repeated. Their paper has two halves: a diagnostic Monte Carlo Tree Search (MCTS) over the skip/repeat program space, and the POLAR predictor (a lightweight network that outputs a program in a single shot, replacing per-input search) itself. The diagnostic search shows that better programs almost always exist, among other findings. This paper’s primary contribution is a careful reproduction of these findings alongside a public release of the code and search implementation Li et al. (2026) do not themselves provide. On this basis, we build a detailed analysis and audit of the discovered programs, together with a brain-inspired analogy for how such a program coordinates computation (fig. 1). POLAR belongs to a broader family of cheap, training-free interventions that recover a model’s latent capability without touching the weights (section 5). Models can be “under-elicited”: they compute a correct answer internally but fail to state it. A POLAR is one concrete way to recover such an answer, and the rest of this paper asks how reliably that recovery works and how recovered programs behave. Concretely, this paper makes three contributions:

Table 1: Every finding in section 4, checked against Li et al. (2026)’s own study. ✓ = reproduced, ⚡ = not reproduced (we tested it and got a contradicting result), ★ = a new finding of ours with no equivalent in the original paper to check against. (F1)–(F4) are Li et al.’s own diagnostic Findings 1–4.

Finding	Description	Status
(F0)	Reproducing Li et al.’s baseline numbers	⚡
(F1)	Skip & Repeat combined beats either alone	✓
(F2)	Occam’s razor: most samples have shorter valid programs than a standard forward pass	✓
(F3)	Harder inputs need larger programs (test-time scaling via recurrence)	✓
(F4)	Valid programs concentrate on short, mostly single-recurrence segments	⚡
(F5)	A small menu of programs generalizes across many questions	★
(F6)	A wrong answer close to the ground truth is rescued $\sim 1.5\times$ more often than a distant one	★
(F7)	Programs are fragile: most edits are essential, and fragility depends on model, not difficulty	★
(F8)	Router gives a pass@1 gain over BASE	⚡
(F9)	Router’s pass@5 gain comes from a small, fixed menu, mainly input-independent	★
(F10)	MMLU-Pro rescuing programs overfit to answer position, not content	★

1. We reproduce Li et al.’s diagnostic MCTS study and its router across 5 models, reconstructing the search algorithm they leave underspecified, since neither it nor its precursor (Li et al., 2025) releases code. We confirm the majority of PoLAR’s diagnostic findings, but cannot reproduce its headline router result at all: in every configuration we test, and across every training recipe and anti-collapse mitigation we tried (section A.8), the router’s top-1 prediction is simply the standard forward pass, never a genuine alternative program (table 1).
2. Building on our reproduction, we audit quantity and quality of the discovered programs, which also helps explain the router’s collapse. Its candidate pool does contain real, executable diversity, but it never ranks the right candidate at the top, always the identity program instead. A small menu of programs already covers most of what a far larger search space finds. These programs are also fragile: undoing a single edit inside a program that fixes a wrong answer breaks the fix more often than not.
3. We develop a neuroscience-motivated analogy between programs-of-layers and thalamo-cortical coordination between cortical-area-like transformer layers, reviewed against the wider brain-inspired-machine learning (ML) literature.

We publicly release code and the full set of MCTS-discovered programs for every question studied here, along with further visualizations such as an interactive program-tree explorer, at <https://dataxis.github.io/RE-PoLar/>.

2 Motivation

2.1 Biological relevance

Brain neurons inspired the earliest artificial neural networks McCulloch & Pitts (1943); Rosenblatt (1958), and neuroscience supplied foundational ideas for the architectures that followed (Fukushima, 1980; Hopfield, 1982). The two fields have since diverged mostly for engineering reasons: hardware rewards architectures suited to dense matrix multiplication, regardless of biological plausibility (Hooker, 2021). Yet artificial networks still repeatedly exhibit representations and computational motifs resembling biological ones (Dabney et al., 2020; Guclu & van Gerven, 2015; Wang et al., 2021; Sussillo et al., 2015) (see section E for further examples).

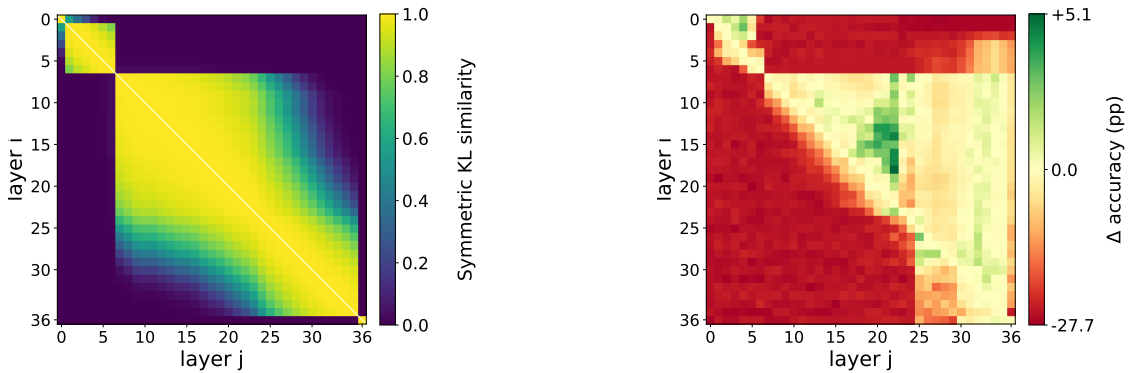
One particularly relevant difference is how computation is organized and routed. Conventional transformers pass every token representation through a fixed sequence of layers in order, but information clearly does not flow this way biologically. Early routing is anatomical, fixed by the sensory modality. Further processing then propagates through progressively higher-order cortical columns, with the thalamus routing information between them. A cortical column, a vertically organized population of neurons, is often treated as a local processing block. Its cell types and connectivity are remarkably conserved throughout the cortex, much

as transformer layers repeat a single form of local computation. There is also evidence that cortical areas built from these columns realize attention-layer-like computation directly (Babu et al., 2026) (section 5 gives the mechanistic detail). Larkum (2013) argues that a column’s architecture and its interaction with the thalamus let the brain integrate context into local computation and distribute the context-sensitive output around the cortex. This parallels how contextual information modulates attention over token embeddings across transformer blocks. How the brain selects which information reaches which columns is still under investigation (Munn et al., 2026; Zolnik et al., 2026). But cortical computation clearly executes in parallel across columns, with only a relevant subset active at any time.

Li et al. (2026)’s program-of-layers (POLAR) caught our attention for exactly this reason. Their *skip* and *repeat* operations let a large language model (LLM) omit or revisit entire layers at inference time without retraining, a complementary form of conditional computation. Specifically, POLAR’s predictor stood out for playing a *thalamus-like* coordinating role (fig. 1): it decides which modules engage and how information flows between them, rather than computing locally itself (see section 5 for further analogy and limitations). Recently, Cox et al. (2026) report that extensive task practice remodels neural circuitry to escape a default, effortful “frontal bottleneck”: further evidence that a practiced, selectively routed path, not one fixed circuit, is the biological norm. Parallel to biological computation under a constrained budget (Gilad et al., 2018), Li et al. find that harder inputs need increasingly long programs.

2.2 Preliminary Evidence for Layer Redundancy

Before finding Li et al. (2026), we asked ourselves: do layers of a pretrained LLM tolerate rerouting at inference time, or could such rerouting even improve it? We were prompted by a blog report that *repeating* a contiguous layer block (re-running it immediately after itself, no retraining) can raise accuracy (Ng, 2026). Independently, Sun et al. (2024) measure cosine similarity between layer activations and find a related pattern: early and final layers are distinct, while middle layers share a common representation space, a split echoed again and again, most recently on vision transformers (Jacobs et al., 2026).



(a) Symmetric Kullback–Leibler (KL) similarity between layer i ’s and layer j ’s output token distributions.

(b) Accuracy Δ (pp) from re-running block $[i, j]$ a second time (repeat, upper triangle) or removing it (skip, lower triangle).

Figure 2: Simply repeating a well-chosen middle block (layers 18–22), no retraining, no other intervention, already improves Qwen3-8B’s accuracy on a 6-domain subset of MMLU-Pro (Wang et al., 2024). The representational pattern in (a) and the behavioral pattern in (b) agree on two boundaries: layer 6, where mixing layers from before and after collapses accuracy under both repeat and skip, and layer 25, past which layers grow mutually redundant, so skipping there turns from harmful to roughly neutral.

Figures 2a and 2b test this directly on Qwen3-8B, on a 6-domain subset of MMLU-Pro (Wang et al., 2024). Figure 2a measures pairwise similarity across all 37 layer-states (the embedding output, plus the output of each of the 36 decoder layers) via symmetric KL divergence. Figure 2b separately sweeps every contiguous layer block $[i, j]$ under *repeat* (re-running a block immediately after itself) and *skip* (removing it). Layer 6 is

the critical boundary in both. Blocks that mix layers from before and after layer 6 collapse accuracy under both repeat and skip, matching a sharp similarity boundary at the same layer in (a). The same pattern holds again past layer 25: layers there are mutually similar in (a) and can again be combined into a contiguous segment, so skipping them in (b) is tolerated, with a well-chosen middle block (best: layers 18–22) even net positive under repeat. Overall, which layers can be combined into a contiguous skip or repeat segment in (b) closely follows which layers share a similar distribution in (a).

This single-model, single-benchmark result already shows naive repeat/skip interventions moving accuracy in a model-specific, structured way. It is an informal, single-operator instance of the question Li et al. formalize and that we reproduce and extend: given a library of layer functions, what is the best per-input program over them, and can systematic search, or a lightweight learned predictor in its place, recover the remaining gain?

3 Methodology

3.1 Program-of-Layers Formalism

A pretrained large language model (LLM) computes its output for a sequence of length T through a fixed sequence of D transformer layers. Each layer is a function $f_i : \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^{T \times d}$ with $i = 0, \dots, D-1$, hidden dimension d . A per-input program-of-layers (POLAR) $\pi = (i_1, \dots, i_K)$ replaces this fixed order with an arbitrary finite index sequence, inducing the composition $F_\pi = f_{i_K} \circ \dots \circ f_{i_1}$. We call π *valid* for a given input if F_π produces a correct prediction. Multiple distinct valid programs can exist for the same input. Simply applying $F_{\text{id}} := f_{D-1} \circ \dots \circ f_0$ is the standard forward pass, which we call BASE when reporting results. We call π a *rescuing* program for a given input if F_{id} is not valid but F_π is.

An arbitrary index sequence π of any length K is intractable to search over directly, so Li et al. (2026) restrict this space of possible programs to a small, structured subset. The D layers are partitioned into contiguous segments by a boundary mask $\mathbf{z}^{\text{seg}} \in \{0, 1\}^D$, where a 1 starts a new segment (each segment is at most $K_{\text{max}}=4$ layers long). Every segment is assigned one operation from $\{\text{skip}, \text{keep}, \text{repeat}\}$ (label vector $\mathbf{z}^{\text{op}} \in \{\text{skip}, \text{keep}, \text{repeat}\}^D$, one entry per segment start): **skip** removes the segment, **keep** runs it once as usual, and **repeat** runs it twice. Section 3.2 describes how Monte Carlo Tree Search (MCTS) explores this operation set. Section 3.3 describes how the router is trained to predict it.

3.2 Search Procedure through MCTS

We describe POLAR’s search procedure in more detail than either source paper Li et al. (2025; 2026) does. Both leave it substantially underspecified, and neither publishes code for it. We reconstruct the search independently and specify previously unstated choices precisely below. Section A.6 argues for each one in detail and reports the evidence behind it.

Search nodes as edit sets. We depart from a literal action-by-action reading of $\pi = (i_1, \dots, i_K)$ for tractability: a node is a *set of edits* on top of the identity program $\text{id} = (0, \dots, D-1)$. An edit $e = (s, \ell, o, \rho)$ places operation $o \in \{\text{skip}, \text{repeat}\}$ (**keep** is not an explicit edit, since every layer not covered by an edit is kept by default) on the contiguous block $[s, s+\ell)$, $\ell \leq K_{\text{max}}$, where $\rho \geq 2$ is the block’s total execution count under **repeat**. A node N is a set of pairwise non-overlapping edits, root $N_0 = \emptyset$. Its program $\text{Prog}(N)$ tiles every gap between edits with **keep** segments and is therefore always complete and directly executable. Because the same edit set is reachable via more than one placement order, we canonicalize nodes by their sorted edit tuple and cache them in a transposition table, which turns the search graph into a directed acyclic graph (DAG) rather than a tree (see section A.6 for details).

Control flow. Each simulation descends from the root, choosing between two moves at every node it visits: while the node still has room under its widening cap $k(v)$ (defined below), search expands it directly with a new child; once capped, search instead selects the child that currently maximizes its Upper Confidence Bound (UCB) score (also defined below) and descends into it. Unlike classical MCTS, there is no separate rollout phase: search scores the node it lands on directly, executing its program with exactly one model call, rather than simulating forward with a default policy until a terminal state. We batch these per-program model calls

across an entire search round rather than evaluating one at a time (section A.7). Reward is 1 if the model’s output is correct and 0 otherwise. That reward backs up along the exact chain of nodes just visited, and a global counter V increments once per simulation, feeding every node’s UCB explore term equally. Section A.6 gives the full pseudocode (algorithms 1 and 2) and the formal definitions of the expansion rule behind this summary. Figure 3 traces one simulation end to end on a toy tree.

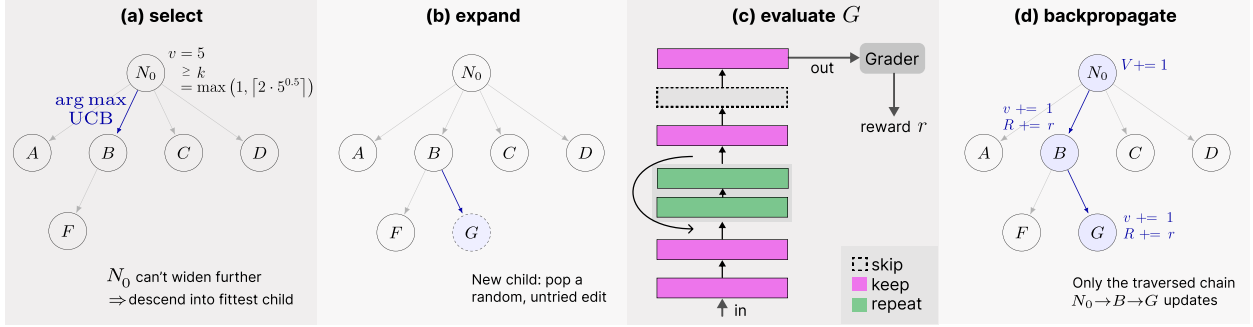


Figure 3: One simulation of algorithm 1, traced end to end on a toy tree ($D=7$). (a) N_0 is already widening-capped, so selection descends by $\arg \max$ UCB into child B rather than expanding N_0 further. (b) B is still under its own cap (1 of 2 children allowed at $v=1$), so **SELECTEXPAND** expands it directly, popping a random untried edit to create G . (c) x is scored under G ’s edit set ($\{\text{skip}[5, 6), \text{repeat}[2, 4) \times 2\}$), producing reward r . (d) r backpropagates along the traversed chain $N_0 \rightarrow B \rightarrow G$, and the global counter V increments once.

Selection: UCB score. We use the same UCB equation as Li et al. to rank a capped node’s children: each child’s score trades off how well it has scored so far against how rarely it has been tried, so search neither fixates on one early success nor keeps re-visiting a branch it already understands well, with a further penalty against candidates that make the executed program longer:

$$\text{UCB}(N) = \underbrace{\frac{R(N)}{v(N)}}_{\text{exploit}} + c \underbrace{\sqrt{\frac{\ln V}{v(N)}}}_{\text{explore}} - \underbrace{\lambda \frac{|\pi_N|}{D}}_{\text{length penalty}}, \quad k(v) = \max(1, \lceil \alpha v^\beta \rceil), \quad (1)$$

where $R(N)/v(N)$ is the cumulative reward over visits per node, c the exploration factor, V the total simulation count across the whole search and $|\pi_N| = D - \sum_{\text{skip } e \in N} \ell_e + \sum_{\text{repeat } e \in N} (\rho_e - 1) \ell_e$ is the executed program length that, together with λ , penalizes long programs. $k(v)$ is a progressive-widening cap absent from Li et al.: a node may only be expanded with a new child while $|\text{children}(N)| < k(v(N))$, which forces the tree to explore programs with more than one edit rather than exhausting single-edit siblings of the root.

3.3 Router: Architecture and Training-Target Construction

Li et al. (2026) replace per-input MCTS for finding a valid program with a single-shot predictor. This router outputs a program representation ($\mathbf{z}^{\text{seg}}, \mathbf{z}^{\text{op}}$) given the input. It first encodes the input with a *frozen* embedding model (Qwen3-Embedding-0.6B (Qwen Team, 2025)), then cross-attends learnable per-layer query embeddings against the resulting token representations to obtain input-conditioned per-layer representations. These pass through a 2-layer transformer encoder for global context across depth. Two linear heads produce segmentation logits ($\mathbb{R}^D$) and operation logits ($\mathbb{R}^{D \times 3}$). Training loss is segmentation binary cross-entropy plus a masked operation cross-entropy (operation loss computed only at segment starts), supervised directly on valid MCTS-discovered programs. Section A.8 traces every architecture and training-data choice above to its source in Li et al.’s own released code. At inference, per-layer segmentation probabilities are thresholded into segment boundaries, and each resulting segment is capped at K_{max} layers. A small beam search over segment operations then ranks the surviving candidate programs before mapping the winner to an executed program. This router adds only $\approx 2.1\text{M}$ parameters (0.01–0.06% of the base model), and per Li et al.’s own reported figure, it costs $\approx 0.8\%$ extra compute over a single forward pass. We reproduce this architecture,

loss, and decoding procedure essentially as Li et al. describe it. We now turn to its training data, which similarly involves a large, separate set of design choices.

Every valid program for a question becomes its own training example, capped at 50 per question. Segmentation and operation targets are read directly off each program’s own $\text{Prog}(N)$ segments (section 3.2), except that – unlike the search itself – **keep** segments are here also capped at $K_{\max}$, matching section 3’s original per-segment bound. This matches what the router’s own decoding enforces at inference: a predicted **repeat** always decodes to exactly two executions, so any MCTS-discovered program whose repeat count differs from two is dropped from training entirely (*Drop*), matching Li et al.’s own behavior; section A.8 compares this against two alternative treatments. Each example carries a scalar weight: 1.0 by default, except a question’s own identity program is down-weighted to 0.3 whenever a strictly shorter valid program also exists for that question. Every example is further scaled by $1/n_{\text{prog}}$ for its question, so questions with many discovered valid programs do not dominate the loss. Section A.8 traces each of these numbers to its source; most are not stated in the original paper’s own text at all.

4 Experiments and Results

This section reports our Monte Carlo Tree Search (MCTS) and router reproduction of Li et al. (2026) (section 4.1), and checks that reproduction against their own diagnostic findings and headline router claim (sections 4.2 and 4.4). It then extends the reproduction with program-level analyses of our own that have no program-of-layers (POLAR) equivalent (sections 4.3 and 4.5). Table 1 lists our findings [F0](#)–[F10](#), checked against Li et al.’s own study.

4.1 Experimental Setup

Models. We evaluate 6 pretrained, instruction-tuned large language models (LLMs): Qwen1.5-MoE-A2.7B-Chat, Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct, LLaMA-3.2-3B-Instruct, Qwen3-8B, and Qwen3-32B. All 6 are used fully frozen throughout this paper. Li et al.’s own study covers four of those; Qwen2.5-7B-Instruct and Qwen3-32B are our own additions. Our MCTS and router training/evaluation track further excludes LLaMA-3.2-3B (justification in section A.5).

Data. In-distribution: DART-Math (Tong et al., 2024), split into five difficulty levels DM-1 (easiest) through DM-5 (hardest) (section A.1 explains the construction), 2,000 queries per level (1,250 train / 250 val / 500 test), 10,000 total. Out-of-distribution: MMLU-Pro (Wang et al., 2024) ($n=12,032$, multiple-choice, up to 10 lettered options per question), used both as a zero-shot transfer target for the DART-Math-trained router and, separately, as its own in-domain MCTS/router track (section 4.4). We also use two arithmetic transfer targets, ASDiv (Miao et al., 2020) ($n=2,305$) and MAWPS (Kadlčík et al., 2023) ($n=520$).

Metrics. BASE ($\tau=0$) is the unmodified model under greedy decoding. BASE (sampling) draws k stochastic decodes at temperatures $\tau \in \{0.3, 0.7, 1.0\}$ and reports the best result across temperatures for each k , following Li et al.. Then, $\text{pass}@k$ is the probability that at least one of the top- k candidates is correct. For BASE (sampling), the k candidates are k stochastic decodes; for the router, they are its own top- k beam-searched candidate programs.

Evaluation protocol. [F0](#) ⚡ Our reproduction of Li et al.’s own BASE numbers does not land uniformly: the gap against their published tables (tables 2 and 7’s Δ columns) is model-dependent, not systematic, and stable from $\tau=0$ through $\text{p}@5$. We keep their LLM generation cap of 50 tokens, but use a different prompt and no chat template throughout, a deliberate deviation that fixes two failure modes we found. However, this is not the reason our numbers differ from theirs (checked thoroughly in sections A.2 and A.3). A likely remaining contributor is that our reconstructed DART-Math split (section A.1) is not guaranteed to match Li et al.’s original, unreleased one. ASDiv/MAWPS use the same protocol. Apart from those, MMLU-Pro is scored as forced-choice over answer-letter logits with no generation step (section A.4). Unlike Li et al., we report hardware and GPU-hours by pipeline stage (section B.1).

Table 2: (F0) (F8) The reproduction gap against Li et al. (2026)’s own numbers is model-dependent (colored Δ = ours – their matching tables), and ROUTER p@1 equals BASE ($\tau=0$) exactly on every model and difficulty: the router always predicts identity. Accuracy (%) by DART-Math difficulty: BASE $\tau=0$ /p@1/p@5 and ROUTER p@1/p@5 are pass@ k on the 500-question held-out test split per difficulty; MCTS is the share of samples that have at least one valid program, computed over the full 2,000-question pool.

Model			DM-1	DM-2	DM-3	DM-4	DM-5
Qwen1.5-MoE-A2.7B	BASE	$\tau=0$	22.6 (-15.4)	24.2 (-0.4)	21.2 (+0.6)	18.0 (+2.8)	12.2 (-2.0)
		p@1	21.6 (-13.8)	24.2 (+2.2)	21.2 (+6.6)	18.2 (+5.6)	11.4 (+4.8)
		p@5	34.4 (-5.6)	36.2 (+10.6)	30.8 (+12.2)	28.6 (+13.6)	21.2 (+9.4)
	MCTS		73.0 (-0.2)	71.0 (+11.3)	67.8 (+17.4)	63.2 (+25.0)	43.9 (+11.5)
	ROUTER	p@1	22.6 (-21.2)	24.2 (-2.0)	21.2 (-0.8)	18.0 (+0.8)	12.2 (-3.2)
		p@5	35.6 (-26.4)	37.4 (-6.6)	34.2 (+1.2)	30.6 (+5.2)	19.0 (-4.2)
Qwen2.5-3B	BASE	$\tau=0$	33.0 (+11.0)	29.2 (+15.6)	24.2 (+13.2)	16.2 (+8.4)	8.6 (+3.4)
		p@1	33.0 (+8.8)	31.4 (+15.4)	24.8 (+13.2)	16.0 (+7.8)	9.4 (+4.0)
		p@5	41.2 (-1.0)	37.0 (+6.8)	33.6 (+13.2)	24.0 (+8.2)	15.4 (+2.4)
	MCTS		82.7 (-4.7)	82.2 (+5.7)	72.6 (+7.6)	68.5 (+17.3)	43.5 (-1.0)
	ROUTER	p@1	33.0 (-11.4)	29.2 (+4.8)	24.2 (+4.0)	16.2 (+4.4)	8.6 (-4.2)
		p@5	45.4 (-14.4)	44.8 (+4.2)	38.6 (+10.4)	29.0 (+11.0)	15.0 (-7.8)
Qwen2.5-7B	BASE	$\tau=0$	50.4	50.4	40.8	29.0	18.4
		p@1	50.2	50.4	41.2	29.4	19.0
		p@5	58.8	57.6	49.6	38.4	25.2
	MCTS		89.0	88.2	81.2	75.5	50.6
	ROUTER	p@1	50.4	50.4	40.8	29.0	18.4
		p@5	64.4	65.2	53.0	42.8	25.6
Qwen3-8B	BASE	$\tau=0$	48.6 (+7.0)	45.8 (+17.0)	33.8 (+14.8)	28.2 (+16.2)	9.6 (-3.4)
		p@1	48.8 (+11.8)	47.6 (+24.6)	34.6 (+25.4)	27.6 (+18.0)	10.0 (+3.0)
		p@5	54.0 (+5.6)	53.8 (+26.0)	42.2 (+28.6)	35.2 (+21.8)	13.0 (+2.4)
	MCTS		88.8 (-2.5)	86.3 (+4.1)	77.8 (+10.7)	71.7 (+18.1)	47.2 (+1.5)
	ROUTER	p@1	48.6 (+5.4)	45.8 (+15.4)	33.8 (+13.0)	28.2 (+14.7)	9.6 (-5.3)
		p@5	60.6 (+5.0)	57.4 (+12.6)	46.2 (+20.8)	37.0 (+18.2)	17.0 (-0.8)
Qwen3-32B	BASE	$\tau=0$	63.2	51.6	37.8	16.6	5.6
		p@1	60.0	48.2	35.6	17.4	5.2
		p@5	74.6	66.0	52.8	34.0	14.4
	MCTS		97.1	96.0	91.5	85.9	58.8
	ROUTER	p@1	63.2	51.6	37.8	16.6	5.6
		p@5	75.0	65.8	64.2	37.6	18.0

4.2 MCTS Diagnostic Findings vs. PoLar

Li et al. (2026)’s own diagnostic MCTS study already establishes what a frozen model’s layers can do beyond the standard forward pass. What follows checks each of those findings against our own MCTS reproduction (5 models, DART-Math Diff 1–5), and reports what we find beyond them. Section C provides additional results.

(F1) ✓ skip and repeat combined beat either alone, on every model and every DART-Math difficulty level, with repeat the stronger of the two mechanisms on its own. This is the same ordering Li et al. report. Table 16 reports BASE < Skip-only < Repeat-only < Skip&Repeat accuracy for each of our 5 models at every DART-Math difficulty level. The Skip&Repeat gain over BASE is positive everywhere.

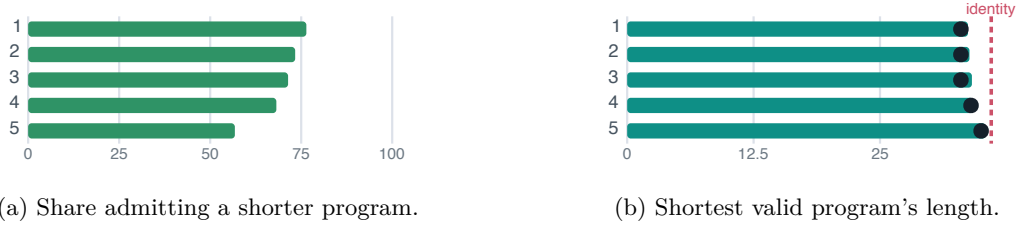


Figure 4: **(F2)** Occam’s razor on Qwen3-8B, by DART-Math difficulty: (a) the share of all questions admitting a program shorter than the standard forward pass; (b) that shortest program’s actual executed length, bar = mean, black dot = median, dotted line marks the identity length D . Same measurement as figs. 13 and 14, all 5 models, in the appendix.

(F2) ✓ (Occam’s razor). Most valid programs are shorter than the standard forward pass, clearly confirming Li et al.’s own finding on 4 of our 5 models and closely on the fifth. Figure 4 shows this directly for Qwen3-8B. Figures 13 and 14 give the full per-model breakdown. On the same 4 models, we even beat Li et al.’s numbers, some by a wide margin and with Qwen1.5-MoE-A2.7B being the exception. This is also their own weakest model on this finding, so this shortfall may not count against reproduction.

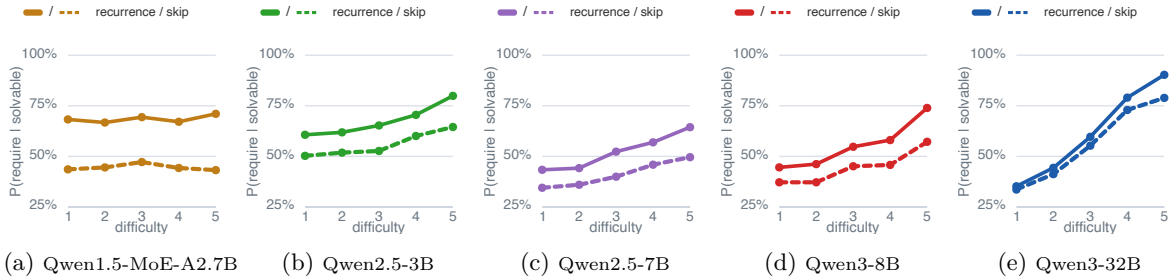


Figure 5: **(F3)** Of every question that is solvable at all, the share that needed recurrence (solid) or needed skipping (dashed) to be solved. We are using the same per-model panel as Li et al. (2026) for our reproduction.

(F3) ✓ Harder inputs rely more on the **skip/repeat** program space, and allowing more test-time recurrence steps gives access to more valid programs, confirming Li et al.’s finding. Figures 10b and 11 confirm the recurrence-budget part directly: on every model, the share of questions with at least one valid program rises monotonically as more recurrence steps are allowed. Figure 5 confirms the difficulty part: the share of solvable questions that specifically need recurrence rises from the easiest to the hardest difficulty on all 5 models. The share that need skipping rises on 4 of 5, with Qwen1.5-MoE-A2.7B again showing outlier behavior. Figure 6 makes the same pattern concrete at the layer level for Qwen3-32B: easier questions solve with skips concentrated in a few positions, while harder questions increasingly rely on repeats spread across all layers. Some layer positions are consistently preferred for an edit and others consistently avoided across all 5 models (figs. 17 and 18 in the appendix). Section 2.2 finds the same avoided region independently for Qwen3-8B (layer 6), visible again here as the same early band that is rarely touched.

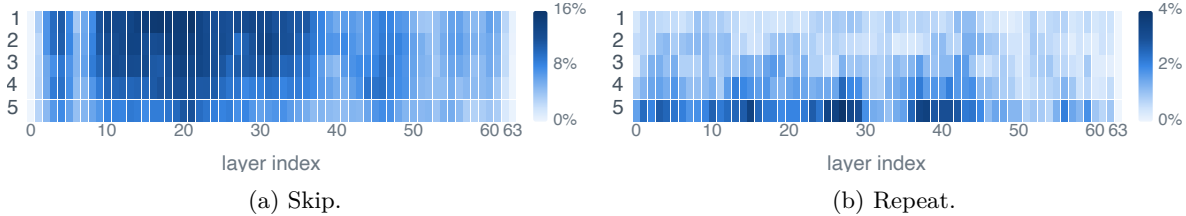


Figure 6: **(F3)** Qwen3-32B: share of shortest-valid programs that **skip** (a) or **repeat** (b) each layer, by layer index and DART-Math difficulty. Easier difficulties concentrate more on skipped layers; harder difficulties spread repeats across more of the stack. Same measurement as figs. 17 and 18, all 5 models, in the appendix.

(F4) ⚡ Valid programs’ segments are not as short as Li et al. find: most of our models fall below both of their length thresholds, sometimes by a wide margin. Li et al. report that 54.5% of segments in valid programs touch only a single layer, and over two-thirds touch at most two. Table 17 reports the same two shares for our 5 models (shortest-valid-program segments, pooled over all difficulties), but 4 of 5 fall below both thresholds. Even the closest model, Qwen1.5-MoE-A2.7B, only partially matches Li et al.. Figure 7a below gives the full segment-length distribution behind these shares. The other sub-claim, that segments are repeated at most once, holds much more broadly: pooling the “0” and “1” extra-repetition bins in fig. 7b, 73–86% of segments across all 5 models use at most one recurrence.

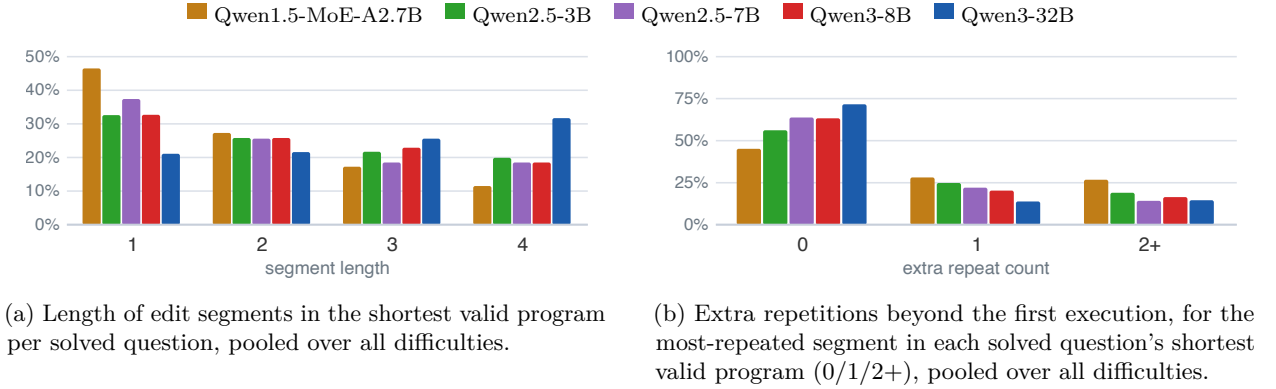


Figure 7: **(F4)** Program-editing structure by segment length, grouped by model.

4.3 Beyond PoLar: Menu Generalization, Error Analysis, and Program Robustness

(F5) ★ A small, fixed menu of non-identity programs already covers most questions: the same handful of programs generalizes across many different questions, rather than every question needing its own bespoke program. Li et al.’s diagnostic study never checks this. For each model, we rank every program by how many questions it solved during search, then run the top- K of those programs against every question in the pool. Figure 8(a) pools that coverage across all 5 models as K grows from 1 to 100 (fig. 12 gives the full per-model/per-difficulty curves). Identity remains the single highest-coverage entry, and every edit below it is a strictly narrower specialization that helps a smaller slice of questions. Figure 8(b) shows the top-25 programs themselves are structurally diverse, editing many different layer bands rather than converging on one recipe: each edits exactly one contiguous layer block, with edits clustering away from an early band around layer 6 and a later one around layer 25.

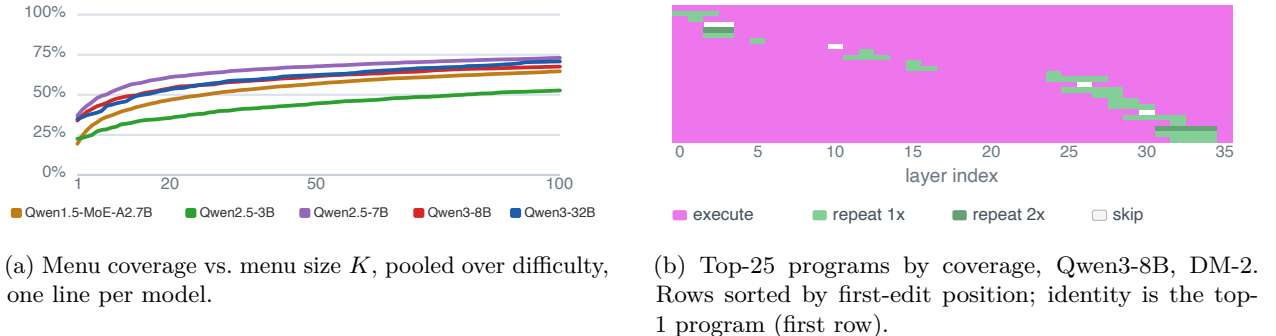


Figure 8: **(F5)** Coverage and visualization of the menu behavior.

(F6) ★ It matters a lot how close a wrong answer already is to the ground truth: a *related* wrong answer (numerically or structurally close to ground truth) is rescued by some valid program roughly 1.5× more often than an *unrelated* one, in every model. We reach this using MiniMax-M2.7-AWQ as an LLM-assisted

classifier for how close each wrong answer is to the ground truth, applied to every DART-Math question the identity program answers wrong, across all 5 models and 5 difficulties (section B.2 gives the full taxonomy, methodology, and error-category/closeness-rescue tables).

(F7) ★ Program fragility is model-dependent, not difficulty-dependent: for every genuine rescuing program (averaging 2.01 edited segments, model-dependent from 1.79 to 2.08), we revert one edited segment at a time back to **keep** and check whether the reward survives. Figure 9 plots the resulting break rate: it is roughly flat across DM-1–DM-5 within every model, ranging 52.5% (Qwen3-32B) to 72.3% (Qwen1.5-MoE-A2.7B) pooled over difficulty. Most edits inside a working program are individually essential, and which model it is predicts fragility far more than how hard the question is.

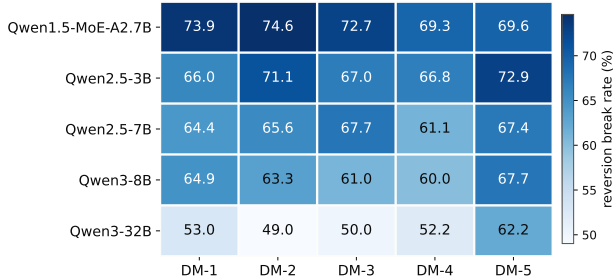


Figure 9: **(F7)** Each cell is the share of rescuing programs (identity wrong, this program right) whose reward breaks when one edited segment is reverted to **keep**, pooled over all single-segment reversions.

Model	Examples	Identity %	Keep %
Qwen1.5-MoE-A2.7B	3,430	7.0%	80.1%
Qwen2.5-3B	6,306	4.2%	83.8%
Qwen2.5-7B	8,519	5.0%	80.7%
Qwen3-8B	8,573	4.4%	83.7%
Qwen3-32B	10,614	2.4%	89.5%

Table 3: **(F8)** Router training-data composition, averaged across all 5 DART-Math difficulty splits, 1,250-question train slice, showing identity examples are a small minority, not a majority-class artifact. *Examples* = program-examples, one per valid MCTS-discovered program, capped at 50/question; *Identity %* = share of the identity program; *Keep %* = share of **keep** operations, across every example

4.4 Router Reproduction: Identity-Collapse

(F8) ⚡ Li et al. (2026)’s headline router claim is a real, if modest, pass@1 gain over BASE (41.6% → 43.2%, +1.6pp), growing to +14.0pp by pass@5. We do not reproduce the pass@1 finding. On every model, difficulty split, and training-data treatment tested, the router’s top-1 decode is always the literal identity program, both for our own reimplement and for Li et al.’s own unmodified training code run on our data (tables 2 and 9). Therefore, we do not evaluate the trained router on our Out-of-Distribution (OOD) transfer targets. Section A.8 documents a systematic sweep of every anti-collapse lever we or Li et al.’s own code expose: segmentation-threshold calibration, class-weighted and focal losses, down-weighting the identity program’s training loss to zero, length-preference reweighting, and a **keep**-probability penalty. None breaks it without a real accuracy cost, nor is it a majority-class artifact of the training data (table 3: identity examples are only 2.4–7.0% per model). Hence, the identity-collapse we find is not an artifact of an under-tuned reproduction, but a robust property of the setup itself.

(F9) ★ Li et al.’s pass@5 gain does replicate (tables 2 and 9), but it comes from beam search diversity. The router never places a correct non-identity program at rank 1, only somewhere in its top-5 (section A.8 has the full evidence chain, grading methodology, and candidate-pool diversity breakdown). Furthermore, table 12 shows that the router’s top-5 predicted programs collapse to a fixed menu of few programs only.

4.5 Separate MMLU-Pro Track

Beyond the DART-Math track above, we also ran a dedicated MCTS and router training directly on MMLU-Pro. The finding below has no equivalent in Li et al.

(F10) ★ Reshuffling which option sits behind which letter (content unchanged) shows MCTS-discovered rescuing programs substantially overfit to the correct answer’s *position*, not its content. Reshuffled rescuing accuracy falls *below* identity’s own, while an identity control confirms the base model itself is close to shuffle-invariant (section D.1). Li et al. report that router-predicted programs retain their gains under OOD cross-domain evaluation. A dedicated, in-domain MMLU-Pro router instead shows the same pass@1

identity-collapse as DART-Math, with a pass@5 margin that is real (section D.2): evidence against genuine content-tracking. We hypothesize the router instead converges on a small, fixed menu of programs that together cover the answer distribution better than 5 programs sampled at random, without needing to read each question’s content (section D.2).

5 Related Work

Saving compute. One line of work removes layers permanently, the same way for every input: LayerDrop (Fan et al., 2020) trains a model so arbitrary layer subsets can be dropped at inference, LaCo (Yang et al., 2024) merges consecutive layers via weight arithmetic, and ShortGPT (Men et al., 2025) drops layers ranked as redundant by input-output similarity. Sun et al. (2024) find the same pattern in language models, probing a similar skip/reorder/parallel-execution space to the one we investigate in section 2; Balef et al. (2026) find it again outside language models, in tabular foundation models. A second line keeps every layer but decides, per input, which ones to run: early-exit classifiers stop once confident (Xin et al., 2020; Zhou et al., 2020), LayerSkip (Elhoushi et al., 2024) shares one such classifier across layers with self-speculative decoding, and FlexiDepth (Luo et al., 2025), MindSkip (He et al., 2024), and GateSkip (Laitenberger et al., 2025) use lightweight routers or residual gates to skip interior layers, still aimed at spending less compute, not unlocking more.

Reusing compute. A third line reuses layers instead of removing them: Universal Transformers (Dehghani et al., 2019) apply one shared block recurrently with a per-token halting policy, looped transformers (Fan et al., 2025) repeat a block for a fixed or learned number of iterations to improve length generalization, MoEUT (Csordás et al., 2024) sparsifies the shared block with mixture-of-experts to scale parameter count without the cost of always running all of it, and Inner Thinking Transformer (Chen et al., 2025) combines adaptive loops with a residual “thinking” stream and per-token routing. Mixture-of-Depths (Raposo et al., 2024) and Mixture-of-Recurions (Bae et al., 2025) route at the token level in a similar spirit, but still need architectural change and training. Unlike all of these, program-of-layers (POLAR) (Li et al., 2026) reuses a frozen, pretrained model’s own layers with no architectural change, training a router to predict a whole program upfront, replacing per-input search with a single forward pass. DR.LLM (Heakl et al., 2025) also learns routing policies from Monte Carlo Tree Search (MCTS) supervision, but locally, per layer, *during* the forward pass, so it cannot represent the multi-layer block repetitions POLAR’s (and our) programs use. As part of our reproduction, which confirms POLAR’s central finding but not all of it (our router reproduction never matches its published numbers, and section 4 surfaces failure modes it does not report), we release all code and data. A separate, adjacent line unlocks a *fixed* model’s latent capability without retraining through inference-time interventions on the input or embedding space, rather than the execution path: random-token embedding injection (Devansh, 2026), random soft prompts (Kim et al., 2026), meaningless-token activation redistribution (Shi et al., 2025), filler-token hidden computation (Pfau et al., 2024), prompt engineering (Liu et al., 2023), and lightweight adaptation via Low-Rank Adaptation (LoRA) (Hu et al., 2022).

Brain-like compute. Babu et al. (2026) show that a cortico-thalamic circuit can realize multihead self-attention, with one head per cortical area and the thalamus computing and routing queries, keys, and values across them. Koenig & Negrello (2026) independently propose transformer operations as a framework for laminar cortical computation. A program-of-layers plays the coordinating role these accounts assign to the thalamus: deciding which areas to engage, in what order, and when to loop, with the POLAR (Li et al., 2026) predictor standing in for the thalamus itself. To our knowledge this specific reading is new, and it is the main way this paper connects to neuroscience. AlKhamissi et al. (2025) similarly partition transformer layers into modules with specialized cognitive networks, starting from pretrained layers but fine-tuning their weights into these modules, unlike our own routing on top of a pretrained model’s frozen layers; section E reviews further such brain-inspired machine learning (ML) approaches. Both approaches coordinate computation *globally*, over the full path rather than a single layer’s decision, in the spirit of the shared, bandwidth-limited channel of Goyal et al. (2021). Mixture-of-experts routing looks similar but stays local, deciding per layer and per token.

6 Conclusion

We revisited the program-of-layers (POLAR) idea proposed by Li et al. (2026) (i.e., input-dependent execution of transformer layers) and replicated its core finding: dynamically skipping or repeating layer operations reliably outperformed the standard, fixed-depth forward pass. However, we failed to replicate the main result: across every model and setting tested, the router’s top prediction consistently collapsed back to the default forward pass, even though the Monte Carlo Tree Search (MCTS) found a better program for nearly every solvable question. Our broader analysis of program structure and robustness suggests that this failure is architectural rather than an artifact of training. Because the error surface over program space is highly fractured, single-shot deterministic prediction is a poor strategy. This echoes how the brain resolves difficult computations, by engaging multiple pathways concurrently rather than committing to a single route prematurely, and opens a concrete direction for widening the program space itself (section 6.1).

6.1 Limitations and Future Work

A primary limitation of our study is that we could not evaluate the trained router on Out-of-Distribution (OOD) data, only BASE, as its top prediction always collapsed to the default forward pass. A router that weighs several candidate programs at once instead of committing to one, resembling the brain’s approach, could address both the collapse and the untested OOD gap. Secondly, several of our MCTS-based findings could not reproduce Li et al. (2026)’s numbers precisely even when matching their setup closely. Beyond closing these two gaps, several other directions remain. Splitting attention and Multi-Layer Perceptron (MLP) sub-layers into independent operators would provide a finer-grained action space. It could also uncover gains similar to those Team et al. (2026) report from treating attention as a separate residual stream. Moving from one static program per input to per-token or per-segment routing decisions is a natural extension. Crucially, our operator set represents only a fraction of biological routing; extending the program space with a parallel-layer execution operator on the same input is a promising direction. Finally, it is unclear whether individual layers play a similar functional role across different backbone models, an analogue at the layer level to cross-model concept neurons (Dravid et al., 2023).

Acknowledgments

We thank Walter Senn (University of Bern) for discussions on (amongst others) the cortico-thalamic view of self-attention. Our work is funded by the Federal Ministry of Research, Technology and Space under grant number 01GQ2505A and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – FIP-12 – Project-ID 528483508. Responsibility for the content of this publication lies with the author.

References

- Ahsan Adeel and M. Bilal. Scalable machines with intrinsic higher mental-state dynamics. *arXiv preprint arXiv:2603.13453*, 2026.
- Badr AlKhamissi, C. Nicolò De Sabbata, Greta Tuckute, Zeming Chen, Martin Schrimpf, and Antoine Bosselut. Mixture of cognitive reasoners: Modular reasoning with brain-like specialization. *arXiv preprint arXiv:2506.13331*, 2025.
- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(3):235–256, 2002.
- Vasanth Kumar Babu, Arno Granier, Timothée Proix, Marmaduke Woodman, Viktor Jirsa, Titas Balvocius, Ausra Saudargiene, and Walter Senn. Cortico-subcortical multi-head self-attention as a substrate for cognitive performance. *bioRxiv*, 2026. doi: 10.64898/2026.08.06.743205. URL <https://www.biorxiv.org/content/early/2026/08/10/2026.08.06.743205>.
- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoung Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic

-
- recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Amir Rezaei Balef, Mykhailo Koshil, and Katharina Eggersperger. Is one layer enough? understanding inference dynamics in tabular foundation models. *arXiv preprint arXiv:2605.06510*, 2026.
- Vincent-Pierre Berges, Barlas Oğuz, Daniel Haziza, Wen-tau Yih, Luke Zettlemoyer, and Gargi Ghosh. Memory layers at scale. *arXiv preprint arXiv:2412.09764*, 2024.
- Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- Yilong Chen, Junyuan Shang, Zhenyu Zhang, Yanxi Xie, Jiawei Sheng, Tingwen Liu, Shuohuan Wang, Yu Sun, Hua Wu, and Haifeng Wang. Inner thinking transformer: Leveraging dynamic depth scaling to foster adaptive internal thinking. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- Adrien Couëtoux, Jean-Baptiste Hoock, Nataliya Sokolovska, Olivier Teytaud, and Nicolas Bonnard. Continuous upper confidence trees. In *Learning and Intelligent Optimization (LION 5)*, pp. 433–445, 2011.
- Patrick H. Cox, Clara A. Scholl, Marissa L. Laws, Nelson E. Jaimes, Xiong Jiang, and Maximilian Riesenhuber. Extensive experience remodels neural task circuitry to escape the frontal bottleneck and increase automaticity of categorization. *Journal of Cognitive Neuroscience*, 2026. DOI: 10.1162/JOCN.a.2618.
- Róbert Csordás, Kazuki Irie, Jürgen Schmidhuber, Christopher Potts, and Christopher D. Manning. MoEUT: Mixture-of-experts universal transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Will Dabney, Zeb Kurth-Nelson, Naoshige Uchida, Clara Kwon Starkweather, Demis Hassabis, Rémi Munos, and Matthew Botvinick. A distributional code for value in dopamine-based reinforcement learning. *Nature*, 577(7792):671–675, January 2020. ISSN 1476-4687. doi: 10.1038/s41586-019-1924-6. URL <http://dx.doi.org/10.1038/s41586-019-1924-6>.
- Luke Darlow, Ciaran Regan, Sebastian Risi, Jeffrey Seely, and Llion Jones. Continuous thought machines. *arXiv preprint arXiv:2505.05522*, 2025.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.
- Devansh. The cheapest way to make open weight AI models better. Blog post, Artificial Intelligence Made Simple, 2026. <https://www.artificialintelligencemadesimple.com/p/the-cheapest-way-to-make-open-weight>. Accessed 2026-08-27.
- Amil Dravid, Yossi Gandelsman, Alexei A Efros, and Assaf Shocher. Rosetta neurons: Mining the common units in a model zoo. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1934–1943. IEEE, 2023.
- Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1909.11556.
- Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped transformers for length generalization. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.15647.

-
- Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, April 1980. ISSN 1432-0770. doi: 10.1007/bf00344251. URL <http://dx.doi.org/10.1007/BF00344251>.
- Ariel Gilad, Yasir Gallero-Salas, Dominik Groos, and Fritjof Helmchen. Behavioral strategy determines frontal or posterior location of short-term memory in neocortex. *Neuron*, 99(4):814–828.e7, August 2018. ISSN 0896-6273. doi: 10.1016/j.neuron.2018.07.029. URL <http://dx.doi.org/10.1016/j.neuron.2018.07.029>.
- Anirudh Goyal, Aniket Didolkar, Alex Lamb, Kartikeya Badola, Nan Rosemary Ke, Nasim Rahaman, Jonathan Binas, Charles Blundell, Michael Mozer, and Yoshua Bengio. Coordination among neural modules through a shared global workspace. *arXiv preprint arXiv:2103.01197*, 2021.
- U. Guclu and M. A. J. van Gerven. Deep neural networks reveal a gradient in the complexity of neural representations across the ventral stream. *Journal of Neuroscience*, 35(27):10005–10014, July 2015. ISSN 1529-2401. doi: 10.1523/jneurosci.5023-14.2015. URL <http://dx.doi.org/10.1523/JNEUROSCI.5023-14.2015>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.
- Jeff Hawkins, Marcus Lewis, Mirko Klukas, Scott Purdy, and Subutai Ahmad. A framework for intelligence and cortical function based on grid cells in the neocortex. *Frontiers in Neural Circuits*, 12:121, 2019.
- Shwai He, Tao Ge, Guoheng Sun, Bowei Tian, Xiaoyang Wang, and Dong Yu. Router-tuning: A simple and effective approach for enabling dynamic-depth in transformers. *arXiv preprint arXiv:2410.13184*, 2024.
- Ahmed Heakl, Martin Gubri, Salman Khan, Sangdoo Yun, and Seong Joon Oh. Dr.LLM: Dynamic layer routing in LLMs. *arXiv preprint arXiv:2510.12773*, 2025.
- Sara Hooker. The hardware lottery. *Communications of the ACM*, 64(12):58–65, 2021.
- J J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, April 1982. ISSN 1091-6490. doi: 10.1073/pnas.79.8.2554. URL <http://dx.doi.org/10.1073/pnas.79.8.2554>.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Mozes Jacobs, Thomas Fel, Richard Hakim, Alessandra Brondetta, Demba Ba, and T Anderson Keller. Block recurrent dynamics in vision transformers. In *International Conference on Learning Representations*, volume 2026, pp. 51617–51642, 2026.
- Marek Kadlčík, Michal Štefánik, Ondřej Sotolář, and Vlastimil Martinek. Calc-x and calcformers: Empowering arithmetical chain-of-thought through interaction with symbolic systems. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 12101–12108, 2023.
- Kohitij Kar, Jonas Kubilius, Kailyn Schmidt, Elias B. Issa, and James J. DiCarlo. Evidence that recurrent circuits are critical to the ventral stream’s execution of core object recognition behavior. *Nature Neuroscience*, 22(6):974–983, April 2019. ISSN 1546-1726. doi: 10.1038/s41593-019-0392-5. URL <http://dx.doi.org/10.1038/s41593-019-0392-5>.
- Heejun Kim, Seungpil Lee, Jewon Yeom, Jaewon Sok, Seonghyeon Park, Jeongjae Park, Taesup Kim, and Sundong Kim. From noise to diversity: Random embedding injection in LLM reasoning. *arXiv preprint arXiv:2605.11936*, 2026. “Random Soft Prompts”; unoptimized random embedding vectors match trained soft-prompt accuracy on math reasoning.
- Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning (ECML)*, pp. 282–293, 2006.

-
- Peter Koenig and Mario Negrello. The neuroscience of transformers. *arXiv preprint arXiv:2603.15339*, 2026. Proposes transformer operations as an orienting framework for laminar cortical computation.
- Filipe Laitenberger, Dawid Kopiczko, Cees G. M. Snoek, and Yuki M. Asano. What layers when: Learning to skip compute in LLMs with residual gates. *arXiv preprint arXiv:2510.13876*, 2025. GateSkip.
- Matthew Larkum. A cellular mechanism for cortical associations: an organizing principle for the cerebral cortex. *Trends in Neurosciences*, 36(3):141–151, March 2013. ISSN 0166-2236. doi: 10.1016/j.tins.2012.11.006. URL <http://dx.doi.org/10.1016/j.tins.2012.11.006>.
- Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? test-time depth adaptation of pretrained llms, 2025. URL <https://arxiv.org/abs/2507.07996>.
- Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? learning program-of-layers in LLMs. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=pl10b6EQAN>. Project: <https://github.com/tianyi-lab/PoLAr>.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys*, 55(9):1–35, 2023.
- Xuan Luo, Weizhi Wang, and Xifeng Yan. Adaptive layer-skipping in pre-trained LLMs. *arXiv preprint arXiv:2503.23798*, 2025. FlexiDepth.
- Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, December 1943. ISSN 1522-9602. doi: 10.1007/bf02478259. URL <http://dx.doi.org/10.1007/BF02478259>.
- Xin Men, Mingyu Xu, Qingyu Zhang, Qianhao Yuan, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. ShortGPT: Layers in large language models are more redundant than you expect. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 20192–20204, 2025.
- Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing english math word problem solvers (ASDiv). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 975–984, 2020.
- Brandon R. Munn, Eli J. Müller, Christopher J. Whyte, Toon Brouwer, Yohan J. John, Jaan Aru, Mototaka Suzuki, Yuri B. Saalman, Matthew E. Larkum, and James M. Shine. Neural selection in corticothalamocortical loops: Heterogeneity as a computational resource. *Progress in Neurobiology*, 264:102945, September 2026. ISSN 0301-0082. doi: 10.1016/j.pneurobio.2026.102945. URL <http://dx.doi.org/10.1016/j.pneurobio.2026.102945>.
- David Noel Ng. Llm neuroanatomy: How i topped the llm leaderboard without changing a single weight. Blog post, March 2026. <https://dnhkng.github.io/posts/rys/>.
- Jacob Pfau, William Merrill, and Samuel R. Bowman. Let’s think dot by dot: Hidden computation in transformer language models. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2404.15758; filler tokens (“.....”) enable hidden multi-token computation.
- Russell A. Poldrack, Fred W. Sabb, Karin Foerde, Sabrina M. Tom, Robert F. Asarnow, Susan Y. Bookheimer, and Barbara J. Knowlton. The neural correlates of motor skill automaticity. *The Journal of Neuroscience*, 25(22):5356–5364, June 2005. ISSN 1529-2401. doi: 10.1523/jneurosci.3880-04.2005. URL <http://dx.doi.org/10.1523/JNEUROSCI.3880-04.2005>.
- Qwen Team. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

-
- F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. ISSN 0033-295X. doi: 10.1037/h0042519. URL <http://dx.doi.org/10.1037/h0042519>.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024.
- Shaan Shah and Meenakshi Khosla. Representational alignment across model layers and brain regions with multi-level optimal transport. *arXiv preprint arXiv:2510.01706*, 2025. MOT.
- Zeru Shi, Yingjia Wan, Zhenting Wang, Qifan Wang, Fan Yang, Elisa Kreiss, and Ruixiang Tang. Meaningless tokens, meaningful gains: How activation shifts enhance LLM reasoning. *arXiv preprint arXiv:2510.01032*, 2025.
- Qi Sun, Marc Pickett, Aakash Kumar Nain, and Llion Jones. Transformer layers as painters. *arXiv preprint arXiv:2407.09298*, 2024.
- David Sussillo, Mark M Churchland, Matthew T Kaufman, and Krishna V Shenoy. A neural network that finds a naturalistic solution for the production of muscle activity. *Nature Neuroscience*, 18(7):1025–1033, June 2015. ISSN 1546-1726. doi: 10.1038/nn.4042. URL <http://dx.doi.org/10.1038/nn.4042>.
- Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, Yutian Chen, Junjie Yan, Ming Wei, Y. Zhang, Fanqing Meng, Chao Hong, Xiaotong Xie, Shaowei Liu, Enzhe Lu, Yunpeng Tai, Yanru Chen, Xin Men, Haiqing Guo, Y. Charles, Haoyu Lu, Lin Sui, Jinguo Zhu, Zaida Zhou, Weiran He, Weixiao Huang, Xinran Xu, Yuzhi Wang, Guokun Lai, Yulun Du, Yuxin Wu, Zhilin Yang, and Xinyu Zhou. Attention residuals, 2026.
- Yuxuan Tong, Xiwen Zhang, Rui Wang, Ruidong Wu, and Junxian He. DART-Math: Difficulty-aware rejection tuning for mathematical problem-solving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Viet Anh Khoa Tran, Emre Neftci, and Willem A. M. Wybo. Contrastive consolidation of top-down modulations achieves sparsely supervised continual learning. *arXiv preprint arXiv:2505.14125*, 2025.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pp. 5998–6008, 2017.
- Peter Y. Wang, Yi Sun, Richard Axel, L.F. Abbott, and Guangyu Robert Yang. Evolving the olfactory system with machine learning. *Neuron*, 109(23):3879–3892.e5, December 2021. ISSN 0896-6273. doi: 10.1016/j.neuron.2021.09.010. URL <http://dx.doi.org/10.1016/j.neuron.2021.09.010>.
- Xiao-Jing Wang. Probabilistic decision making by slow reverberation in cortical circuits. *Neuron*, 36(5): 955–968, December 2002. ISSN 0896-6273. doi: 10.1016/s0896-6273(02)01092-9. URL [http://dx.doi.org/10.1016/s0896-6273\(02\)01092-9](http://dx.doi.org/10.1016/s0896-6273(02)01092-9).
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2201.11903>.
- Ji Xin, Raphael Tang, Jaeeun Lee, Yaoliang Yu, and Jimmy Lin. DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 2246–2251, 2020.

Yifei Yang, Zouying Cao, and Hai Zhao. LaCo: Large language model pruning via layer collapse. *arXiv preprint arXiv:2402.11187*, 2024.

Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. BERT loses patience: Fast and robust inference with early exit. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Timothy A. Zolnik, Britta J. Eickholt, Zoltán Molnár, and Matthew E. Larkum. The layer 6b theory of attention. *Neuron*, 114(4):565–582, February 2026. ISSN 0896-6273. doi: 10.1016/j.neuron.2025.11.024. URL <http://dx.doi.org/10.1016/j.neuron.2025.11.024>.

A Reproduction Details

A.1 Dataset construction: our DART-Math split

Tong et al. (2024) and Li et al. (2026) both report results by difficulty level, but neither releases the discrete difficulty split itself. We reconstruct this split ourselves from the underlying DART-Math data and describe the construction below, so that our difficulty-wise numbers are reproducible even though Li et al.’s original split is not. Our construction matches Li et al.’s reported level counts, but we cannot guarantee it reproduces their exact split. We do not redistribute the resulting split directly: the MATH half of the underlying pool traces back to the same competition-math problems as `hendrycks/competition_math`, now delisted from HuggingFace amid a disputed-ownership claim over the source content, so re-hosting a derived copy ourselves would just repack the same disputed content once more; the GSM8K half has no such issue. Instead, our released code includes a single script that deterministically rebuilds this exact split from the public DART-Math pool, available at <https://github.com/DATEXIS/RE-PoLar>.

To build the pool, we use the following four HuggingFace datasets: the MATH query pool (`hkust-nlp/dart-math-pool-math`, giving question, ground-truth answer, and level, with 7,473 distinct queries) and its per-query fail rates (`hkust-nlp/dart-math-pool-math-query-info`), and the corresponding GSM8K query pool (`hkust-nlp/dart-math-pool-gsm8k`, 7,468 distinct queries) and fail rates (`hkust-nlp/dart-math-pool-gsm8k-query-info`). Fail rate is defined as $1 - \text{pass rate}$, where the pass rate is the value Tong et al. measure from sampling DeepSeekMath-7B-RL. We join each pool with its query-info table on query id, concatenate the MATH and GSM8K records into a single pool, and sort the result ascending by fail rate, breaking ties by query id for determinism.

We then cut the sorted pool into 5 near-equal quantile bins, from easiest to hardest, and treat these bins as Li et al.’s DM-1 (easiest) through DM-5 (hardest). Within each bin we apply a seeded shuffle (seed 42, per level) and then take the first 1,250 queries for train, the next 250 for val, and the next 500 for test, giving 2,000 queries per level and 10,000 in total, matching the $5 \times 2,000$ counts reported in Li et al.’s Appendix D.1, and capping each level well below the full labeled pool’s 14,941 queries across the 5 levels.

A.2 Prompt sensitivity in evaluation

Initially, we adopted Li et al. (2026)’s own DART-Math evaluation protocol verbatim, which we call `polar_default` throughout this paper. In their Appendix D.4, models are instructed to answer with “...formatted strictly as `\boxed{ANSWER}`,” under greedy, 50-token-capped generation. The word inside the braces however is a literal placeholder, not an empty template. This section documents two failure modes that follow from this and how we fix both. This is not unexpected: large language model (LLM) evaluation is known to be highly sensitive to prompt-formatting choices that are semantically irrelevant to the underlying task (Sclar et al., 2024).

Two failure modes. We call the first `prompt template echo`: the model emits `\boxed{ANSWER}` itself, verbatim, instead of a real answer (also catching `$_\boxed{ANSWER}$`, `\boxed{\text{answer}}`, and case variants). The DART-Math extraction code we use, unmodified from Li et al.’s own, always takes the *last* `\boxed{...}` span in the response, so a later echo is exactly what determines the grade when one occurs. On Qwen3-8B, 43.8% of BASE’s generations under this exact wording are an echo. A second, mechanistically distinct failure produces no `\boxed{}` span at all for a different reason: the model reasons in free-form prose and exhausts the 50-token budget before ever writing `\boxed{}`, rather than echoing the placeholder (this holds even for the Qwen3 models, where thinking mode is explicitly disabled for every generation in this paper). This is the same `unboxed` failure mode section B.2 later reports for Qwen3-32B. It dominates `polar_default` on 3 of our 6 models, each with a near-zero echo rate. Table 4 reports `unboxed` rate and `prompt template echo` rate as separate columns because accuracy alone conflates the two failure modes.

A prompt-format sweep. A defect this large was worth checking for a fix that does not change the answer format itself, and for how much it varies across models. We swept 6 candidate prompt wordings across all 6 models in our study on our DART-Math split (section A.1, 10,000 samples), reporting accuracy, `unboxed` rate,

and `prompt template echo` rate for each. We chose these 6 starting from Heakl et al. (2025)’s own DR.LLM protocol: DR.LLM (Heakl et al., 2025) is one of the closest related works to program-of-layers (POLAR) itself (section 5), also learning Monte Carlo Tree Search (MCTS)-supervised routing policies, so testing its exact prompt wording is the natural comparison point, rather than one we invented ourselves. Response prefill and a single worked demonstration (introduced below) came out of an earlier, smaller pilot sweep as the two most effective interventions, so alongside `polar_default` and DR.LLM’s own faithful wording, the final sweep also carries DR.LLM’s chat-native prefill and one-shot variants, combined and separately. Table 4 gives the full result, table 5 the exact templates; the table’s own caption also covers a grading detail specific to `polar_oneshot`.

Both fixes work almost everywhere. `polar_oneshot` was designed to stay as close as possible to Li et al.’s own wording: it keeps the exact same instruction and format, only adding one trivial worked example (“What is 1+1?”) in the same style. Response prefill and the single trivial demonstration both push the `unboxed` rate down from 91.2% to effectively 0% on Qwen3-8B, at no extra token cost beyond our existing 50-token budget. Both interventions are a significant improvement over `polar_default` (McNemar’s test, $p < 0.001$) on 4 of 6 models: Qwen2.5-3B, Qwen2.5-7B, Qwen3-8B, Qwen3-32B. On Qwen1.5-MoE-A2.7B only `polar_oneshot` is significant ($p < 10^{-14}$); `drllm_chat_prefill` is not ($p = 0.06$, numerically slightly *worse* than `polar_default`). Two models are exceptions, for different reasons. LLaMA-3.2-3B: neither intervention significantly improves on `polar_default`. Qwen3-32B: `polar_oneshot` is a large, significant gain (+34.3pp, $p < 0.001$), but only a partial fix; its `unboxed` rate falls from 98.2% to still 38.6%. Overall, `polar_oneshot` stays close to Li et al.’s original wording (appropriate for a reproduction) while fixing the majority of the format-compliance failures: the `unboxed` rate falls substantially for Qwen3-32B and the Qwen2.5 models, and the `prompt template echo` rate falls from 43.8% on Qwen3-8B to effectively 0% across the board. Both failure modes are fixed, not eliminated, under this tradeoff. Section B.2 tags what remains in more depth, including the `unboxed`/`prompt template echo` split used there.

Consequence for our program analysis. Left unaddressed, `polar_default` would make a large share of “rescuing programs” on Qwen3-8B specifically just formatting fixes for the echo defect. Better still, `polar_oneshot` improves accuracy relative to `polar_default` on every model except a tie on LLaMA-3.2-3B, which we drop from the MCTS and router track anyway (section A.5), so we use it as the generation prompt throughout our program-level analysis, on every model. Still, Qwen3-32B carries a residual risk the others do not: even under `polar_oneshot`, 38.6% of its generations still contain no `\boxed{}` span, so format-compliance failures may still contaminate its rescuing-program counts, reflected in the error-category breakdown in section B.2.

A.3 Decision against chat templates

Beyond prompt *content* (as discussed before in section A.2), a separate, largely orthogonal axis is prompt *mechanism*: whether the model’s native chat template is applied at all. We ablate this directly, holding content fixed at `polar_oneshot` on both sides, on a 200-question pooled sample (seed 42, DART-Math) per model (table 6). Applying the chat template never improves accuracy and typically costs a substantial amount: the effect is negative on all 6 models. We therefore use no chat templates throughout this paper.

This choice is not merely a convenient default that happens to also be the more faithful one for some models. For LLaMA-3.2-3B specifically, no chat template *is* Li et al. (2026)’s own real mechanism, so our choice and their released mechanism coincide there. For their other models (Qwen3-8B, Qwen2.5-3B, Qwen1.5-MoE-A2.7B), Li et al.’s own code applies a chat template, so our no-chat-template choice is a deliberate deviation from their released mechanism, simply made because we measure it performing better on our data, not because it is easier to implement.

A.4 Full baseline reproduction breakdown

Table 7 extends table 2 with what it omits: p@2–p@4 and LLaMA-3.2-3B (excluded from the main table; section A.5 explains why next), plus the two Out-of-Distribution (OOD) transfer targets and MMLU-Pro, all under `polar_oneshot` with no chat template (sections A.2 and A.3 argue for this choice).

Table 4: Accuracy, `unboxed` rate, and `prompt template echo` rate, all $6 \times 6 = 36$ model/prompt-variant combinations, on our 10,000-query DART-Math split (section A.1). **Bold** = best accuracy for that model. **Grader**: `polar_default`/`polar_oneshot` use PoLar, Li et al. (2026)’s own last-`\boxed{\}`-match DART-Math extraction (byte-identical to their vendored copy); the four `drllm` variants use `mathruler`, Heakl et al.’s own grading library (Heakl et al., 2025) – each graded the way its own source protocol would grade it. `polar_oneshot` repeats the instruction-and-answer block twice inside the 50-token budget, risking a hallucinated third, incomplete `\boxed{\dots}` span that a naive last-match extraction would grab instead of the real answer; we truncate before grading at the first complete span, so it is, in effect, graded on the *first* `\boxed{\dots}` span rather than the last (`polar_default`’s single, unrepeated block never needs this). Superscripts are McNemar’s paired test against that model’s own `polar_default` row: *** $p < 0.001$, * $p < 0.05$, unmarked = not significant. Table 5 gives the exact wording of every variant swept here.

Model	Variant	Grader	Accuracy	<code>unboxed</code>	<code>prompt template echo</code>
Qwen1.5-MoE-A2.7B	<code>polar_default</code>	PoLar	16.3%	0.3%	2.70%
	<code>drllm (faithful)</code>	<code>mathruler</code>	2.0%***	89.2%	0.00%
	<code>drllm_chat_prefill</code>	<code>mathruler</code>	15.6%	0.0%	0.00%
	<code>polar_oneshot</code>	PoLar	19.3% ***	2.0%	0.00%
	<code>drllm_chat_oneshot</code>	<code>mathruler</code>	13.2%***	18.3%	0.00%
	<code>drllm_chat_oneshot_prefill</code>	<code>mathruler</code>	15.3%*	0.0%	0.00%
Qwen2.5-3B	<code>polar_default</code>	PoLar	0.1%	99.5%	0.00%
	<code>drllm (faithful)</code>	<code>mathruler</code>	22.1%***	2.0%	0.00%
	<code>drllm_chat_prefill</code>	<code>mathruler</code>	22.0%***	0.0%	0.00%
	<code>polar_oneshot</code>	PoLar	22.6% ***	1.0%	0.00%
	<code>drllm_chat_oneshot</code>	<code>mathruler</code>	21.6%***	0.0%	0.00%
	<code>drllm_chat_oneshot_prefill</code>	<code>mathruler</code>	21.2%***	0.0%	0.00%
Qwen2.5-7B	<code>polar_default</code>	PoLar	22.7%	42.6%	0.00%
	<code>drllm (faithful)</code>	<code>mathruler</code>	35.0%***	0.0%	0.00%
	<code>drllm_chat_prefill</code>	<code>mathruler</code>	33.7%***	0.0%	0.00%
	<code>polar_oneshot</code>	PoLar	37.0% ***	0.0%	0.00%
	<code>drllm_chat_oneshot</code>	<code>mathruler</code>	35.0%***	0.0%	0.00%
	<code>drllm_chat_oneshot_prefill</code>	<code>mathruler</code>	33.6%***	0.0%	0.00%
LLaMA-3.2-3B	<code>polar_default</code>	PoLar	19.4%	2.4%	0.08%
	<code>drllm (faithful)</code>	<code>mathruler</code>	16.0%***	25.7%	0.00%
	<code>drllm_chat_prefill</code>	<code>mathruler</code>	19.3%	0.0%	0.00%
	<code>polar_oneshot</code>	PoLar	19.2%	0.0%	0.00%
	<code>drllm_chat_oneshot</code>	<code>mathruler</code>	18.8%	1.5%	0.00%
	<code>drllm_chat_oneshot_prefill</code>	<code>mathruler</code>	18.2%*	0.0%	0.00%
Qwen3-8B	<code>polar_default</code>	PoLar	22.1%	0.4%	43.84%
	<code>drllm (faithful)</code>	<code>mathruler</code>	5.1%***	91.2%	0.00%
	<code>drllm_chat_prefill</code>	<code>mathruler</code>	31.6%***	0.0%	0.00%
	<code>polar_oneshot</code>	PoLar	33.7% ***	0.0%	0.02%
	<code>drllm_chat_oneshot</code>	<code>mathruler</code>	30.0%***	6.9%	0.00%
	<code>drllm_chat_oneshot_prefill</code>	<code>mathruler</code>	30.5%***	0.0%	0.00%
Qwen3-32B	<code>polar_default</code>	PoLar	0.5%	98.2%	0.17%
	<code>drllm (faithful)</code>	<code>mathruler</code>	31.2%***	46.4%	0.00%
	<code>drllm_chat_prefill</code>	<code>mathruler</code>	48.8%***	0.0%	0.00%
	<code>polar_oneshot</code>	PoLar	34.8%***	38.6%	1.01%
	<code>drllm_chat_oneshot</code>	<code>mathruler</code>	51.4% ***	0.2%	0.00%
	<code>drllm_chat_oneshot_prefill</code>	<code>mathruler</code>	49.4%***	0.0%	0.00%

Table 8 reports $\text{BASE}(\tau=0)$ accuracy on MMLU-Pro (all 14 official categories, $n=12,032$, scored as forced-choice: a single forward pass’s argmax over answer-letter logits, no generation step) and the two OOD transfer targets, ASDiv (Miao et al., 2020) and MAWPS (Kadlčík et al., 2023), both scored under the identical setup as table 7. Where Li et al. (2026) report matching ASDiv/MAWPS numbers, Δ (in parentheses) gives ours minus theirs; we do not report a matching MMLU-Pro Δ , since Li et al. publish only a per-subject breakdown (13 of the 14 categories we evaluate, excluding computer science), never a single aggregate accuracy.

Every Δ on ASDiv/MAWPS is positive except LLaMA-3.2-3B (both benchmarks) and Qwen1.5-MoE-A2.7B (ASDiv only), the same two models that mostly run below the paper on DART-Math (table 7); the direction

Table 5: The 6 prompt variants swept in table 4 (question text abbreviated as <q>), every message spelled out in full. Rows marked † are chat-templated (section A.3); the literal chat-template markup itself (special tokens etc.) is tokenizer-specific and omitted, but every turn’s exact text content is shown.

Variant	Template
polar_default	“Solve the following math problem and output ONLY the final answer directly, formatted strictly as \boxed{ANSWER}. ### Problem Start <q> ### Problem End Answer:”
drllm (faithful)†	User: “Question: <q> The final answer MUST BE put in \boxed{} and no explanation.”
drllm_chat_prefill†	User: “Question: <q> The final answer MUST BE put in \boxed{} and no explanation.” Assistant: “\boxed{”
polar_oneshot	“Solve the following math problem and output ONLY the final answer directly, formatted strictly as \boxed{ANSWER}. ### Problem Start What is 1+1? ### Problem End Answer: \boxed{2}” “Solve the following math problem and output ONLY the final answer directly, formatted strictly as \boxed{ANSWER}. ### Problem Start <q> ### Problem End Answer:”
drllm_chat_oneshot†	User: “Question: What is 1+1? The final answer MUST BE put in \boxed{} and no explanation.” Assistant: “\boxed{2}” User: “Question: <q> The final answer MUST BE put in \boxed{} and no explanation.”
drllm_chat_oneshot_prefill†	User: “Question: What is 1+1? The final answer MUST BE put in \boxed{} and no explanation.” Assistant: “\boxed{2}” User: “Question: <q> The final answer MUST BE put in \boxed{} and no explanation.” Assistant: “\boxed{”

Table 6: Chat template ablation: BASE($\tau=0$) accuracy with vs. without the model’s native chat template, prompt content held fixed at polar_oneshot, 200-question pooled sample (seed 42) per model. “chat template” adds the model family’s own system message where Li et al. (2026)’s code applies one (Qwen2.5-family, Qwen1.5-MoE-A2.7B); Qwen3-8B’s chat setting has no system message, matching Li et al.’s own polar/eval.py found at (<https://github.com/tianyi-lab/PoLar/blob/main/polar/eval.py>).

Model	no chat template	chat template	Δ (pp)
Qwen1.5-MoE-A2.7B	22.5%	7.0%	−15.5
Qwen2.5-3B	22.0%	20.0%	−2.0
Qwen2.5-7B	44.0%	36.5%	−7.5
LLaMA-3.2-3B	20.5%	2.5%	−18.0
Qwen3-8B	34.0%	33.5%	−0.5
Qwen3-32B	40.0%	1.0%	−39.0

of the DART-Math gap is at least broadly consistent with the OOD gap per model, even though we do not have a mechanistic explanation for either.

A.5 Dropping LLaMA-3.2-3B

LLaMA-3.2-3B’s BASE reproduction does not reproduce cleanly even after our two fixes (sections A.2 and A.3), so we exclude it from our MCTS and router-training/evaluation track. BASE($\tau=0$) accuracy does not fall

Table 7: Full BASE($\tau=0$) and BASE(sampling) p@1–p@5 breakdown (%) on our reconstructed DART-Math split (section A.1), 500 questions/difficulty, all 6 models, **polar_oneshot**, no chat template (section A.3). Colored Δ = ours – Li et al. (2026)’s matching table, where one exists; Qwen2.5-7B and Qwen3-32B are new here (see (F0) for the reproduction-gap pattern).

Model	Metric	DM-1	DM-2	DM-3	DM-4	DM-5
LLaMA-3.2-3B	$\tau=0$	18.8 (-23.6)	14.8 (-13.8)	17.2 (-10.0)	20.0 (-7.6)	24.4 (-4.2)
	p@1	19.2 (-21.4)	14.8 (-13.8)	16.2 (-11.2)	19.0 (-9.0)	24.4 (-4.8)
	p@2	22.6 (-21.4)	18.0 (-17.0)	20.2 (-9.4)	21.2 (-9.2)	28.2 (-4.6)
	p@3	24.6 (-21.6)	21.2 (-17.6)	21.8 (-9.4)	25.0 (-6.4)	32.0 (-2.4)
	p@4	25.2 (-21.8)	23.4 (-18.0)	23.4 (-8.8)	26.4 (-5.8)	34.6 (-0.2)
	p@5	27.2 (-20.4)	25.0 (-18.2)	25.2 (-7.6)	28.0 (-4.8)	35.8 (+0.2)
Qwen1.5-MoE-A2.7B	$\tau=0$	22.6 (-15.4)	24.2 (-0.4)	21.2 (+0.6)	18.0 (+2.8)	12.2 (-2.0)
	p@1	21.6 (-13.8)	24.2 (+2.2)	21.2 (+6.6)	18.2 (+5.6)	11.4 (+4.8)
	p@2	25.8 (-10.6)	29.0 (+4.8)	24.8 (+9.0)	21.2 (+7.6)	14.6 (+6.4)
	p@3	30.2 (-8.6)	31.6 (+7.0)	27.6 (+11.0)	24.2 (+10.2)	17.4 (+7.8)
	p@4	32.4 (-7.2)	35.0 (+10.0)	29.0 (+11.2)	27.4 (+12.6)	19.8 (+8.6)
	p@5	34.4 (-5.6)	36.2 (+10.6)	30.8 (+12.2)	28.6 (+13.6)	21.2 (+9.4)
Qwen2.5-3B	$\tau=0$	33.0 (+11.0)	29.2 (+15.6)	24.2 (+13.2)	16.2 (+8.4)	8.6 (+3.4)
	p@1	33.0 (+8.8)	31.4 (+15.4)	24.8 (+13.2)	16.0 (+7.8)	9.4 (+4.0)
	p@2	37.2 (+4.6)	34.8 (+12.0)	29.4 (+13.4)	19.0 (+6.8)	11.2 (+2.6)
	p@3	39.2 (+1.4)	35.4 (+8.6)	31.6 (+13.4)	21.0 (+7.2)	13.0 (+3.0)
	p@4	40.2 (-0.4)	36.2 (+7.0)	32.8 (+13.6)	22.4 (+7.2)	14.2 (+3.6)
	p@5	41.2 (-1.0)	37.0 (+6.8)	33.6 (+13.2)	24.0 (+8.2)	15.4 (+2.4)
Qwen2.5-7B	$\tau=0$	50.4	50.4	40.8	29.0	18.4
	p@1	50.2	50.4	41.2	29.4	19.0
	p@2	52.4	53.0	44.8	33.6	20.8
	p@3	56.2	55.4	46.8	35.4	22.2
	p@4	57.8	56.8	48.2	37.2	24.0
	p@5	58.8	57.6	49.6	38.4	25.2
Qwen3-8B	$\tau=0$	48.6 (+7.0)	45.8 (+17.0)	33.8 (+14.8)	28.2 (+16.2)	9.6 (-3.4)
	p@1	48.8 (+11.8)	47.6 (+24.6)	34.6 (+25.4)	27.6 (+18.0)	10.0 (+3.0)
	p@2	50.4 (+8.4)	50.4 (+24.8)	38.6 (+27.0)	31.4 (+20.8)	11.0 (+2.2)
	p@3	52.0 (+6.2)	51.6 (+25.6)	40.2 (+27.8)	32.4 (+21.0)	11.6 (+1.8)
	p@4	53.0 (+6.0)	53.0 (+26.4)	41.4 (+28.8)	33.8 (+21.0)	12.4 (+2.2)
	p@5	54.0 (+5.6)	53.8 (+26.0)	42.2 (+28.6)	35.2 (+21.8)	13.0 (+2.4)
Qwen3-32B	$\tau=0$	63.2	51.6	37.8	16.6	5.6
	p@1	60.0	48.2	35.6	17.4	5.2
	p@2	65.6	53.8	40.8	20.6	8.0
	p@3	70.8	59.2	45.2	25.8	11.2
	p@4	73.0	62.8	48.6	30.8	12.6
	p@5	74.6	66.0	52.8	34.0	14.4

Table 8: BASE($\tau=0$) accuracy (%): MMLU-Pro (overall, all 14 categories, forced-choice logit scoring) and OOD transfer targets ASDiv ($n=2,305$) and MAWPS ($n=520$, generation-based, same prompt/mechanism as table 7), all 6 models. Colored Δ = ours – Li et al. (2026)’s matching table, where one exists.

Model	MMLU-Pro (overall)	ASDiv	MAWPS
LLaMA-3.2-3B	31.4	64.4 (-14.0)	70.6 (-0.9)
Qwen1.5-MoE-A2.7B	24.4	52.7 (-6.4)	62.3 (+20.6)
Qwen2.5-3B	35.2	68.1 (+18.6)	62.9 (+26.7)
Qwen2.5-7B	42.3	75.3	67.9
Qwen3-8B	42.1	78.7 (+11.6)	71.5 (+19.4)
Qwen3-32B	48.7	77.0	77.7

monotonically with nominal difficulty like the other 5 models (table 7, section 4.1), instead rising from its own hardest point back up through DM-5, its own easiest, a pattern Li et al. (2026) report too (their Table 1). It also undershoots Li et al.’s published numbers almost everywhere (tables 7 and 8), and unlike the other three models with a published table, this gap does not shrink under either fix: its placeholder-echo rate was already near zero and its chat mechanism already matched Li et al.’s own released one, so neither applies here. With both confounds ruled out, its BASE reproduction itself simply does not reproduce well, leaving 5 of our 6 models for the rest of this paper, including our own additions Qwen2.5-7B and Qwen3-32B.

Algorithm 1 PROGRAMMCTS(x, y, N_{sim})

Require: input x , ground truth y , layer count D , simulation budget N_{sim}

```
1:  $N_0 \leftarrow \text{NODE}(\emptyset)$  ▷ root = identity program id = (0, ..., D - 1)
2: for  $n = 1$  to  $N_{\text{sim}}$  do
3:    $N, \text{chain} \leftarrow \text{SELECTEXPAND}(N_0)$  ▷ descend/widen by progressive widening, algorithm 2
4:    $\pi \leftarrow \text{Prog}(N)$  ▷ tile  $N$ 's gaps with keep
5:    $r \leftarrow 0$  if  $\pi$  skips every layer, else  $\mathbb{1}\{F_\pi(x) = y\}$  ▷ the only model call this iteration
6:   for  $M \in \text{chain}$  do ▷ every node on the path just taken, root to  $N$ 
7:      $v(M) \leftarrow v(M) + 1$ 
8:      $R(M) \leftarrow R(M) + r$ 
9:   end for
10:   $V \leftarrow V + 1$  ▷ global count, read by every node's UCB explore term
11: end for
12: return  $\{(\text{Prog}(N), R(N)/v(N))\}$  over every visited  $N$  ▷  $R(N)$  can exceed 1 once a node is revisited across simulations, so dividing by  $v(N)$  turns it back into a rate; a program alone would not distinguish valid (rate 1) from invalid
```

Algorithm 2 SELECTEXPAND(N_0)

```
1:  $N \leftarrow N_0$ 
2:  $\text{chain} \leftarrow [N_0]$ 
3: while  $\neg \text{CANEXPAND}(N)$  and  $N$  has children do ▷ capped or exhausted, and there's somewhere to go
4:    $N \leftarrow \arg \max_{C \in \text{children}(N)} \text{UCB}(C)$  ▷ descend deeper, looking for room to expand
5:   append  $N$  to chain
6: end while
7: if  $\text{CANEXPAND}(N)$  then ▷ room under  $k(v(N))$ , eq. (1), true immediately at the root while  $v$  is small
8:    $N \leftarrow \text{EXPAND}(N, \text{RANDOMCHOICE}(\text{untried}(N)))$ 
9:   append  $N$  to chain
10: end if
11: return ( $N, \text{chain}$ )
```

A.6 MCTS design choices: what we changed from a literal reading of the papers, and why

Section 3.2 specifies our search reconstruction precisely; this subsection justifies each place it departs from a literal reading of Li et al. (2026) (ICML version and the authors' own earlier prestudy (Li et al., 2025), published under the pre-rename name *CoLa*), with the evidence behind each choice. Algorithms 1 and 2 give the full pseudocode, built around POLAR's own Upper Confidence Bound (UCB) exploration term (eq. (1)).

Reading the pseudocode. Algorithms 1 and 2 use two helper definitions:

$$\text{CANEXPAND}(N) := \text{untried}(N) \neq \emptyset \wedge |\text{children}(N)| < k(v(N)),$$

and $\text{EXPAND}(N, a)$ returns the transposition-deduplicated child for edit set $N \cup \{a\}$, creating it only if no earlier placement order already has. Algorithm 2 descends past a node only once it is already at its widening cap or has exhausted its actions; a node with room to grow expands immediately instead of being compared against its children, so budget is spent breadth-first near the root and only pushed deeper once shallow nodes saturate. The listing omits our optional ϵ -random-exploration mechanism (off by default, unused throughout this paper's results, covered below).

Progressive widening ($k(v)$ in eq. (1)). Given our action space's size, the root node needs some mechanism that stops it from spending the entire search budget expanding at depth 1. Li et al.'s own ϵ -random exploration mechanism (below) plausibly serves a similar role for a search space this large; we instead default to progressive widening (Couëtoux et al., 2011), a standard technique for large or continuous action spaces (Browne et al., 2012). At the root, with $D=36$ (Qwen3-8B) and $K_{\text{max}}=4$, there are already 276 single-edit actions to try before a *second* edit is even considered, against our own simulation budget of

$N_{\text{sim}}=200$ per question (matched to the preprint’s stated value, see “Other parameters matched to the papers’ stated values” below). Without $k(v)$ capping a node’s children, the root’s untried-action list would never empty within budget: every simulation would add one more single-edit sibling under the root, selection would never descend, and the tree would stay stuck at depth 1, unable to discover the multi-edit programs (e.g. *skip and repeat* together) that motivate (F1) (skip-and-repeat jointly beats either alone). $k(v) = \lceil \alpha v^\beta \rceil$ is the standard *progressive widening* technique, capping branching factor as a slowly growing function of visit count so early search stays shallow while still allowing unbounded eventual depth. We keep $\beta=0.5$, matching that technique’s usual $O(\sqrt{N})$ growth rate. $\alpha=2.0$ has no canonical value in the literature either (domain-tuned everywhere we checked) and is not separately ablated here.

ϵ -random exploration. Algorithm 2 omits this mechanism to avoid implying it is active in the results this paper reports. When enabled ($\epsilon>0$), it sits inside the **while** loop, right after the loop condition (line 3): whenever N has an untried edit and a uniform draw falls below ϵ , N is immediately expanded with a random untried edit and returned instead of continuing to the UCB-argmax descent step (line 4). It substitutes for descending further, and can only trigger at a node that failed CANEXPAND because of the widening cap, not because it ran out of actions. The preprint (Li et al., 2025) states this mechanism as: “the algorithm selects a random unexplored child node with probability 0.1 instead of the one with the highest UCB score ...” For comparison, Heakl et al. (2025)’s own released MCTS implements a related mechanism, substituting a uniformly random *existing* child for the UCB argmax during descent. Our default is $\epsilon=0$.

Any-order edit placement. We allow a node’s edits to be placed in any order, rather than requiring each new edit to start after the previous one ends. An earlier *cursor*-constrained version enforced that ordering and introduced a bias having nothing to do with the model: under a constant reward (isolating search geometry from model signal), 50.0% of placed edits landed in the late third of the layer stack against a 30.4% uniform baseline, and a specific early-then-early two-edit program was $4.3\times$ less likely to ever be evaluated than a specific late-then-late one. We removed this bias by allowing edits at any node in any order, deduplicating equal edit sets via the transposition table below rather than by construction order. We deliberately did not also allow overlapping/nested edits: the router’s output space cannot represent them, so keeping the search space representable by its downstream consumer took priority, even though a literal reading of Li et al.’s Appendix B.3 could permit them.

Transposition table (directed acyclic graph (DAG) instead of a tree) and the reward cache.

A direct consequence of the previous choice: once edits can be placed in any order, the same edit set is reachable via more than one path, so we canonicalize nodes by their sorted edit tuple and cache them rather than letting different placement orders become different nodes. Without this, one program would accumulate visit/reward statistics under several separate identities, splitting the statistics UCB depends on and wasting budget re-discovering programs already evaluated under a different name. A separate reward cache, keyed by the (input, program) pair rather than by node identity, additionally avoids a repeated model call whenever the same executed program recurs for the same input under a *different* edit set (e.g. two edit sets that tile to the same program via *keep*-filled gaps), which the node-level transposition table above does not catch on its own. This reward cache also matters for a second reason, beyond efficiency. Reward grades a greedy-decoded generation against ground truth via exact match on the `\boxed{...}` span (section A.2), and generation runs in batches; since bf16 addition is not associative, the batched-matmul/attention kernel a row executes depends on the shape of the batch it happens to share, and greedy decoding argmaxes at every generated token, so this can occasionally flip which token is emitted, and the flip changes every later step’s conditioning and can cascade into a different final answer, a full 0/1 reward-label flip caused purely by noise. Because the cache reuses a program’s first-computed generation and reward label instead of regenerating it, the same program is never re-batched and re-flipped once it has been evaluated once.

Backpropagation and UCB under multiple parents. Once transposition dedup is in place, a node can legitimately be reached from more than one immediate parent (edit-set $\{A, B\}$ from both $\{A\}$ and $\{B\}$), raising two questions. First, UCB: under our default global- V reading (justified below), N ’s explore term uses the shared global counter, not the specific parent, so N has exactly one UCB value regardless of which parent asks, and multi-parent structure never puts this at risk: everything stays well-defined. Even

under the non-default alternative reading of V as the immediate parent’s own visit count rather than one shared global counter (the next paragraph justifies keeping the global default), no decision is corrupted, since $\arg \max_{C \in \text{children}(P)} \text{UCB}(C)$ only ever compares N against its true siblings under one shared parent P . Second, backpropagation: algorithms 1 and 2 sidestep choosing which parent to propagate through. The chain variable is simply the list of nodes this simulation’s selection loop just walked, and backpropagation replays exactly that list; each chain that reaches N updates $v(N)/R(N)$ independently and additively, like any other node’s statistics. A NODE’s back-pointer field is consequently left unset (used only by an alternative, non-default selection mode where every node has a single parent by construction).

Global V versus the parent’s own visit count. Both Li et al. (2025; 2026) state the UCB explore term’s V as “the total number of simulations”, which we read literally as one counter shared by the whole tree, incremented by every completed simulation (including the root’s own bootstrap), not, as an easy misreading, the specific parent’s own visit count at the point of comparison. The two readings coincide in a plain, single-path tree, but diverge once the DAG structure above lets a node have more than one parent. The local-parent-visits reading is also the textbook one: standard Upper Confidence bounds applied to Trees (UCT) (Kocsis & Szepesvári, 2006) compares children using their shared parent’s visit count, precisely because a global counter breaks UCB1’s (Auer et al., 2002) per-node independent-bandit regret bound (Browne et al., 2012). We keep the global reading anyway: for a paper whose goal is reproducing Li et al.’s findings, matching their literal formula takes priority over a differently motivated regret-bound guarantee we are not claiming in the first place. The textbook-local variant remains available as a configuration flag.

Hierarchical descent versus a flat, whole-tree selection reading. The Selection line in Li et al.’s appendix reads “traverse tree using UCB to reach a *leaf* program” (the preprint (Li et al., 2025) omits “leaf”), which could describe either our hierarchical, per-node UCB descent (algorithm 2) or a flatter rule comparing every still-expandable node in the whole tree directly and expanding whichever wins, regardless of depth. We implemented and ran both: the flat variant produced substantially deeper, near-constant-depth trees on complete production data (median edit-count 10, max 26), sitting awkwardly against (F4) (valid programs predominantly short). We also checked Heakl et al.’s own MCTS implementation (`data_generation.py`) as a possible tie-breaker. Its `MCTSNode.is_leaf()` is `len(self.children) == 0`, so a node can never be selected again once it gets its first child, and a from-scratch reproduction of their exact selection loop (synthetic reward, 20 runs) confirms every node in their tree gets at most one child, a strict linear chain, kept from visibly degenerating only by their search’s own early-exit stopping rule. This rules out DR.LLM’s code as evidence for the flat reading, though the comparison is still informative: our hierarchical mode’s progressive-widening-gated notion of “not yet fully expanded” is the standard UCT definition of *leaf*, which neither DR.LLM’s code nor our flat-selection reading implements. We therefore default to hierarchical descent throughout this paper.

DR.LLM is not directly comparable. There are two further differences from Heakl et al.. First, their search terminates a question as soon as it finds a program both shorter than the current best and correct, rather than exhausting the full simulation budget the way algorithm 1 always does, consistent with the strict, at-most-one-child-per-node chain noted above. Second, their reward for DART-Math questions is a continuous, partial-credit symbolic grader, not the binary correct/incorrect signal Li et al. and we both use (valid iff reward 1, section 3.2). Both differences matter beyond implementation detail. An early-exit search structurally under-counts how many equally good *alternative* programs exist per question (exactly the diversity property section 3.3’s multi-program labelling depends on), and a continuous reward changes what “valid” means, so DR.LLM’s own reported search statistics are not directly comparable to ours or Li et al.’s without accounting for both.

Every tree gets its own random seed. A naïve implementation could share one random stream across every input’s search tree; we instead derive a distinct, deterministic seed per question (a hash of a base seed and the question’s id), so no two inputs’ trees ever make the same sequence of random expansion/tie-breaking choices. This matters: an earlier shared-seed version measurably collapsed search diversity across *different questions*. Per-question seeding is on by default throughout.

Other parameters matched to the papers’ stated values, not tuned. $\lambda=5.0$ (length-penalty weight), the repeat-count cap $\rho-1 \leq 4$ (an edit’s total execution count $\rho \leq 5$, section 3), and the simulation budget $N_{\text{sim}}=200$ per question are all stated explicitly in the preprint’s Appendix B; we match them because the paper states them, not because we tuned them. The global- V backup rule discussed above is likewise adopted because Li et al. state it, not on independent merit; the textbook-UCT alternative remains available precisely because global- V ’s justification here is paper fidelity, not algorithmic superiority. The one constant with no source to match is the UCB exploration constant $c=\sqrt{2}$: neither paper states a value, so we fall back to the standard UCT default (Kocsis & Szepesvári, 2006), a mathematical convention rather than a fidelity or tuning choice.

Taken together. Every departure above is backed by an ablation or a structural argument, and every non-default alternative discussed above remains available as a configuration flag.

A.7 Accelerating MCTS reward computation: masked-batch decoding and its Key-Value (KV)-cached extension

Evaluating a candidate program’s reward during MCTS means generating and grading an answer for every program encountered in a round of tree expansion, each of which may execute a different subset, order, or repetition of layers. A naive implementation evaluates one program at a time: apply the program to the model (physically reordering, skipping, or repeating its layer modules), then run an ordinary cached `generate()` call over that round’s questions. Once a search round has diversified past a handful of trees, distinct programs approach one per tree and per-program question batches collapse to size 1. Generation then becomes launch-overhead-bound rather than compute-bound and severely under-utilizes the GPU.

Masked-batch decoding. This mechanism removes the bottleneck by sharing forward passes across an entire round’s rows regardless of which distinct program each row uses. Rather than mutating the model’s layer list per program, every row’s own layer-execution path is looked up directly. At each decode step and for each transformer layer, all rows whose next required layer matches are gathered into one batched forward call, executed once, and scattered back into a shared per-row residual-stream state, so a layer’s weights are read once per group of co-scheduled rows, not once per row. Grouping uses a greedy largest-group-first schedule: each step buckets rows by their next required layer and fires only the largest bucket before regrouping, trading a little wait time for a few early-ready rows against fewer, larger forward calls. A configurable group-size cap bounds peak memory when many rows converge on the same layer, and each round is split into length-sorted buckets so one unusually long prompt only forces padding on its own bucket.

A KV-cached extension. We originally ran masked-batch decoding without caching, recomputing the full prefix at every decode step; we added a real per-row, per-path-position key/value cache, running the identical algorithm as single-program cached generation (prefill once, then decode one token per step through the cache) while still batching across distinct programs, purely to speed up search further. Because a `repeat` operation revisits the same physical layer at two different positions in a program’s path, each (row, path-position) pair gets its own cache slot rather than one slot per layer, mirroring how our serial execution engine treats each path position as an independent shallow copy of the layer. We validate bit-identical output to single-program cached generation whenever no other row shares a forward call with it (row count 1, including under production left-padding), on different model families and multiple GPU architectures. Its residual disagreement with the serial path once real batching is active is indistinguishable from the batch-shape noise already documented in section A.6.

Validated at production scale. On Qwen3-8B’s complete DART-Math difficulty-1 split (2,000 questions), KV-cached and uncached masked-batch solved counts differ by exactly 1 question (1,776 vs. 1,777) and shorter-than-identity rates by 0.0012, inside the established noise floor, while the KV-cached version is $1.10\times$ faster wall-clock (8h14m vs. 9h04m). This was enough evidence to run every split and every model in this paper’s MCTS with both masked-batch decoding and its KV-cached extension turned on.

Table 9: ROUTER’s real online-execution pass@1–5, Drop-CE recipe, averaged across all 5 DART-Math difficulty splits, all 5 models in this study; the intermediate pass@2–4 ranks here are not in table 2, which reports only pass@1/pass@5 per difficulty level rather than pooled across difficulty. ROUTER’s pass@1 equals BASE($\tau=0$) exactly: it always predicts identity. The pass@5 gap over pass@1 is the real, non-random rescue signal documented in section 4.4, not something ROUTER’s own top-1 choice can access.

Model	pass@1	pass@2	pass@3	pass@4	pass@5
Qwen1.5-MoE-A2.7B	19.6%	21.4%	23.6%	27.9%	31.4%
Qwen2.5-3B	22.2%	25.2%	29.0%	32.1%	34.6%
Qwen2.5-7B	37.8%	40.6%	43.3%	47.4%	50.2%
Qwen3-8B	33.2%	36.3%	38.6%	41.2%	43.6%
Qwen3-32B	35.0%	40.8%	45.2%	48.9%	52.1%

A.8 Router training recipe, architecture, and the identity-collapse investigation

Section 3.3 briefly describes how we turn MCTS output into router training examples, and this subsection traces each architecture and training-data choice to its source. It also documents every anti-collapse lever we tried, and closes with our best mechanistic account of why the collapse resists all of them. Every item below is verified directly against Li et al. (2026)’s own released code at <https://github.com/tianyi-lab/PoLar>. We sweep several data-treatment/loss combinations (“The full recipe sweep” paragraph below defines the full grid) and name each by concatenating the two, e.g. Drop-CE; “Selecting one recipe” picks the single winning combination, Drop-CE, used everywhere else in this paper, including table 9. Unless a paragraph states a model explicitly, every number in the identity-collapse investigation below, up to “Selecting one recipe across models and difficulties,” is Qwen3-8B on DART-Math DM-1; that final paragraph is where the investigation’s conclusions are checked against all 5 models and difficulties.

Evaluation protocol: online execution, not cache-only. POLAR’s evaluation script scores a router’s top- k candidates correct if any is either (a) present in the offline MCTS cache of certified-valid programs for that question, or (b) actually executed online against the target model and found correct, a fallback for candidates the offline search never encountered. Their released online-execution fallback raises an uncaught error whenever reached, so we initially evaluated using only the cache-only path (a); this understates the router, since the offline cache only contains programs a finite-budget search happened to explore and certify. Every result in this section therefore uses online execution, never the cache-only proxy.

Architecture hyperparameters: identical to PoLar’s own defaults, not swept Section 3.3 describes the router’s architecture qualitatively (frozen embedding backbone, per-layer query cross-attention, a small transformer over the layer dimension, two linear heads). Our implementation uses working dimension $d_{\text{model}}=256$, 4 cross-attention/encoder heads, a 2-layer cross-layer transformer encoder (feedforward width $4d_{\text{model}}=1024$), dropout 0.1, and a 512-token question-truncation limit. All five match Li et al.’s own `polar/model.py/polar/train.py` exactly. The frozen encoder is Qwen/Qwen3-Embedding-0.6B (hidden size 1024), projected into d_{model} by a single learned linear layer. Counting only trainable parameters matches Li et al.’s own reported $\approx 2.1\text{M}$. That same section also describes segmentation probabilities being thresholded into segment boundaries at inference, without giving a number: a layer starts a new segment once its predicted segmentation probability reaches 0.5, the code’s own fixed default, and the only value on this list we later vary (*Segmentation-threshold calibration* below, one of the anti-collapse levers). The router’s separate training-time hyperparameters (learning rate, batch size, epoch count), unlike the architecture above, are swept rather than fixed to a single default; *The full recipe sweep* and *Selecting one recipe across models and difficulties* below give the grid and the selection criterion, validation loss, without tabulating every winning value individually.

Multi-program labelling. By default, every valid MCTS-discovered program for a question becomes its own training example, capped at 50 programs per question, matching Li et al.’s own code default exactly; no router-training run in this paper overrides it.

Router training-data handling for repeat segments with multiplicity greater than two. POLAR’s router represents a **repeat** operation as one class among three (skip/keep/repeat) with no encoded repetition count. Its decoder always realizes a predicted **repeat** as exactly two executions of the segment, regardless of how many times the original MCTS-discovered program repeated it. Their released label-construction code only ever matches a **repeat** segment against exactly two consecutive repetitions. Any MCTS-discovered program containing a segment repeated three or more times fails to parse under their grammar and is silently dropped from their training set in its entirety, not merely the offending segment.

Our own MCTS allows repetition counts up to $\rho-1 = 4$ beyond the first execution ($\rho \leq 5$ total executions, again matching Li et al.’s stated search-space bound). A segment repeated more than twice is common in what our search actually finds: a majority of discovered **repeat**-containing programs (53.3%) have at least one segment with a repetition count other than two, echoing the same tension as (F4)’s reproduced at-most-one-recurrence sub-claim (measured there per-segment, pooled across models, rather than per-program here). This directly conflicts with Li et al.’s own reported qualitative finding that valid execution programs “typically require at most a single recurrence per segment.” This is a discrepancy we have not resolved. We flag it as an open question about whether our search dynamics differ from theirs or whether their reported statistic was computed over a different aggregation (e.g. one representative program per input rather than every discovered valid program).

Given a training program contains such a segment, three treatments are possible, and we compare all three under a single, otherwise-identical training and evaluation protocol:

- **Drop.** Discard the entire program from training whenever any **repeat** segment’s count is not exactly two, exactly reproducing POLAR’s own released behavior.
- **Crop.** Retain the program and supervise the segment as an ordinary **repeat** label, ignoring the true count, equivalent to truncating it to two executions without checking whether that shorter program is still correct. This keeps roughly twice as many training programs.
- **Crop and Re-verify.** Truncate the segment to exactly two executions, *re-execute this shortened program online against the real question and ground truth*, and keep it as a training label only if it is still correct, discard it otherwise. Considerably more expensive than the other two treatments, which need no extra model calls.

On Qwen3-8B’s reconstructed DART-Math DM-1 split, re-verification of all 1,105 (sample, program) pairs with a **repeat** count other than two finds that 758 (68.6%) remain correct once cropped to two executions and re-run online against the real question and ground truth, while 347 (31.4%) do not and are discarded entirely, meaning *Crop* is training on a substantial minority of labels that do not actually hold under the router’s own executable semantics.

The full recipe sweep. Alongside this data-treatment axis (Drop/Crop/Crop-and-Re-verify), we also sweep the loss function: standard cross-entropy (CE), Li et al.’s own default, against a class-balanced focal loss, motivated by Heakl et al.’s own use of focal loss to handle the same kind of skip/keep/repeat class imbalance in their router. Every data-treatment/loss combination is itself swept over learning rate ($\{1e-4, 3e-4, 5e-4, 8e-4, 1e-3, 3e-3\}$), batch size ($\{32, 128, 256\}$), and epoch count ($\{3, 10\}$) with the AdamW optimizer, exactly matching Li et al.’s own stated grid ($6 \times 3 \times 2 = 36$ combinations, the “36-combination hyperparameter grid” referenced throughout this section). We name a data-treatment/loss pair by concatenating the two, e.g. Drop-CE; the “Selecting one recipe” paragraph below picks the single winning pair used everywhere else in this paper.

Training-data composition, full breakdown. Table 3 reports per-model averages: *Identity %* is the share of training program-examples that are the identity (full-depth) program, and *Keep %* is the share of per-segment operation labels that are **keep** rather than **skip** or **repeat**. The 50-program cap above applies *per question*, not to a category’s total count across the split: identity, for instance, is just one label among a question’s own capped set, so how often it shows up in total depends on how many different questions it happens to solve, not on the cap itself. Table 10 gives the complete per-difficulty numbers behind it,

Table 10: Router training-data composition, Drop treatment, full per-difficulty breakdown, 1,250-question train slice (data composition depends only on the data treatment, not the loss function). *Solvable* = questions with ≥ 1 MCTS-verified-correct program; *no-id %* = share of those with no valid identity program at all; *examples* = the resulting number of training program-examples; *id %*/*Keep %* as table 3.

Model	DM	Solvable	No-id %	Examples	Id %	Keep %
Qwen1.5-MoE-A2.7B	1	923	68.1%	4,196	7.0%	80.1%
	2	883	68.2%	4,037	6.9%	79.9%
	3	825	74.2%	3,380	6.3%	80.0%
	4	801	67.8%	3,592	7.2%	80.2%
	5	553	73.6%	1,946	7.5%	80.4%
Qwen2.5-3B	1	1,032	60.8%	8,538	4.5%	84.0%
	2	1,034	63.2%	8,183	4.3%	83.9%
	3	891	65.7%	6,593	4.5%	84.0%
	4	876	73.2%	5,754	3.8%	83.6%
	5	543	81.6%	2,464	3.8%	83.6%
Qwen2.5-7B	1	1,114	43.9%	11,042	5.1%	80.8%
	2	1,098	45.3%	11,031	5.1%	80.7%
	3	1,005	54.4%	8,664	5.0%	80.6%
	4	956	58.4%	8,087	4.7%	80.5%
	5	635	68.8%	3,773	5.1%	80.7%
Qwen3-8B	1	1,116	45.3%	12,245	4.4%	83.9%
	2	1,078	47.1%	11,485	4.4%	83.8%
	3	956	57.4%	8,460	4.4%	83.8%
	4	913	58.5%	7,621	4.5%	83.6%
	5	600	76.0%	3,053	4.4%	83.5%
Qwen3-32B	1	1,213	35.8%	15,264	3.0%	89.7%
	2	1,195	45.0%	14,265	2.9%	89.6%
	3	1,148	60.2%	11,559	2.8%	89.5%
	4	1,084	79.5%	8,509	2.0%	89.3%
	5	735	92.1%	3,474	1.4%	89.2%

all 5 models $\times$ all 5 DART-Math difficulties, computed on the real 1,250-question positional train splits. *No-valid-identity %* is the share of solvable questions whose identity program is not even a valid execution (the trigger condition for the anti-collapse **keep**-probability penalty below). It climbs sharply with difficulty for every model, from 36–68% at DM-1 to 68–92% at DM-5, tracking each model’s own accuracy decline. Identity-example and **keep**-label share are comparatively stable within a model across difficulties, but vary meaningfully *across* models (2.0–7.5% identity, 79.9–89.7% **keep**), consistent with the per-model recipe variance documented below (Qwen3-32B’s own preference for Crop over Drop), not a single universal constant.

Segmentation-threshold calibration: rejected, and it isolates where the collapse actually sits.

Sweeping the segmentation threshold introduced above (*Architecture hyperparameters*, 0.5 by default) down to 0.05 finds the segmentation head itself healthy: at the default threshold, essentially every held-out question already has multiple segment boundaries fire (max boundary probability ≈ 0.87). The collapse instead sits entirely in the *operation* head: across every threshold value we tried, its argmax operation is **keep** for effectively every segment ($\geq 95\%$), **skip** for none, regardless of how many segment boundaries the (unrelated) threshold lets through. A lower threshold only changes how many **keep**-labelled segments a program has, not whether any of them are ever anything else, so no threshold value breaks the collapse, and every one still decodes to a program equivalent to identity. This directly motivates the loss-side levers below: if the operation head’s own class predictions are the bottleneck, the fix has to act on that head’s training signal, not on the segmentation decode rule.

Down-weighting identity. When a question has both its identity (full-depth) program and at least one strictly shorter valid alternative among its labelled examples, we down-weight the identity program’s own loss contribution to $0.30\times$ (every other program, including long non-identity ones, stays at $1.0\times$). This directly

implements the paper’s own stated justification. While the paper never gives a number for it, 0.30 is not our own tuning choice: it is the exact value in Li et al.’s own README’s worked training command.

Per-sample weight normalization. Each program-example is additionally scaled by $1/n_{\text{prog}}$ for its question, where n_{prog} is that question’s own program count after the 50-program cap above, so a question with many valid MCTS-discovered programs does not dominate the loss over one with only a few. Same provenance pattern as the down-weight: absent from the paper text, off by default in Li et al.’s own code, and present only in their README’s worked example, which we follow.

keep segments are chunked to K_{max} too. Converting a raw MCTS program into router training targets (segmentation + per-segment operation labels) caps every segment type (including **keep**) at $K_{\text{max}}=4$ layers by default, matching what the router’s decoding procedure enforces at inference. This is a representation choice at the program→target conversion step, not something the search itself imposes: MCTS’s action space (section 3) bounds **skip/repeat** block length only. **keep** is never an explicit search action (it is the identity baseline any edit is placed on top of, section 3.2), so a discovered program can contain an arbitrarily long uninterrupted **keep** run before conversion.

Length-preference reweighting. Li et al.’s code additionally supports a length-preference reweighting policy mode ($\exp(-\beta \cdot |\pi|)$ per example, $\beta \in \{0, 0.02, 0.05, 0.1, 0.2\}$), which exists in our implementation too but which even their own README never demonstrates using. A full sweep of this axis on top of our usual learning rate/batch size/epoch grid, across every data-handling/loss combination above, on Qwen3-8B’s DM-1 split shows every one of the 6 resulting checkpoints’ pass@1 still equals BASE exactly (46.8%). While length preference reshapes *which* programs the router prefers among options it already considers, it does not break the identity collapse.

The anti-collapse keep-probability penalty: real diversity, always at a real accuracy cost. Li et al.’s code exposes a second anti-collapse lever, independent of the identity-program down-weight above: an auxiliary loss term that penalizes the router’s mean predicted **keep**-probability across layers. It only activates when its weight is > 0 , and only over questions whose identity (full-depth) program is not even a valid execution. This targets a different, larger subset of the training data than the down-weight above: on our diff-1 data, 45.3% of questions (805/1,777) have no valid identity program at all, versus 54.7% for the down-weight’s own trigger condition. Their CLI default is 0.0 (off), and their README’s worked command, which supplies the 0.30 down-weight value above, never sets it either.

A 216-configuration sweep (6 values of $\lambda \in \{0, 0.1, 0.3, 0.5, 1.0, 2.0\} \times$ the usual 36-combination hyperparameter grid, Drop-CE, DM-1) is the first result to break the collapse under real, online-execution grading: at $\lambda=1.0$, 3.6% of test questions receive a genuinely non-identity top-1 prediction (real pass@1=49.6% vs. BASE’s 50.8%, -1.2pp ; pass@5 essentially unaffected, 60.8% vs. 61.2%). At $\lambda=2.0$ the penalty is clearly too aggressive: nonidentity rate reaches 40.0% but accuracy collapses outright (-16.4pp). **This is a real, generation-graded break in the collapse, but always at a real accuracy cost, never a net gain:** at every λ tested, router pass@1 is at or below BASE, and rescue@1 (the fraction of BASE-wrong questions the router’s top-1 pick actually rescues) stays 0 even for the checkpoints trained with $\lambda=0.5$ and $\lambda=1.0$ that exhibit real non-identity behaviour. The router becomes willing to deviate from identity without becoming more likely to deviate *correctly*. We return to why next.

Real per-input signal exists in the router’s candidate diversity, but rank-1 never captures it. Forcing identity out of the candidate pool entirely (re-decoding top-6, discarding identity if present, keeping the remaining 5) turns pass@ k into “best of the top- k non-identity candidates.” On DM-1’s identity-wrong subset ($n=133$), accuracy drops sharply this way, to 34.4% for Drop-CE (range across variants 17.6–45.6%). This is surprising: a non-identity, MCTS-verified-correct alternative exists for essentially every question where identity itself is correct (96.3–100.0% across DM-1–5, matching (F2)). The resolution: the router’s identity-excluded candidates rarely land inside the question’s own MCTS-verified valid-program set (Drop-CE: 1.5% cumulative through rank-5, since that recorded set reflects only a finite-budget search, not an exhaustive one), but real online-execution-graded rescue is far higher (29.3% for the same checkpoint), a $\sim 15\text{--}20\times$ gap,

Table 11: Cross-model recipe comparison: validation loss (winning hyperparameter configuration) and real online-execution pass@1/pass@5, averaged across all 5 difficulty splits, for the four comparable variants (Drop/Crop $\times$ CE/focal). Drop-CE is the recipe used throughout the rest of this paper. Drop-CE(pass@5) below is corrected to the real full-500-question test split (same backfill as tables 2 and 9); Crop-CE(pass@5) still uses the original $n=250$ half, not yet backfilled, so that one column is not apples-to-apples with the rest of the table (verified against the repo’s own result files: no full-500 Crop-CE backfill exists yet).

Model	Validation loss		Pass@5	
	Drop-CE	Crop-CE	Drop-CE	Crop-CE
Qwen1.5-MoE-A2.7B	1.071	1.086	31.4%	30.5%
Qwen2.5-3B	1.024	1.034	34.6%	32.7%
Qwen2.5-7B	1.088	1.105	50.2%	49.6%
Qwen3-8B	1.008	1.032	43.6%	44.9%
Qwen3-32B	0.892	0.915	52.1%	56.4%

holding at every difficulty level. Most real rescues come from candidates MCTS never sampled but which genuinely execute correctly, so the router’s candidate diversity is real and carries genuine executable signal. What it cannot do is *rank* that signal: rescue@1 stays at or near 0 for every checkpoint, at every difficulty, while rescue@5 is substantial. If the router’s confidence score correlated at all with correctness, some rescuable questions should land specifically at rank-1; instead essentially none do. This is not a train/test artifact: the same MCTS-membership check on a same-size slice of *training* data gives 0.8–1.6% cumulative membership, barely different from test’s 1.5–2.3%, and real rescue@1 on that same train slice is likewise exactly 0. **Even on data it was directly trained on, the router’s top-1 pick, when identity fails, is never the correct rescuing program.** This is consistent with (though not conclusively proven by) positive-only supervision: training data is built exclusively from MCTS’s own recorded valid programs, with no negative signal from the programs it recorded as invalid, or from visit-count/UCB preference. The router’s rank ordering plausibly reflects proximity to the collapsed, *keep-heavy* training marginal rather than per-candidate correctness, since nothing in its training signal penalizes a wrong candidate for scoring above a right one.

Selecting one recipe across models and difficulties. Table 11 extends the treatment $\times$ loss comparison to all 5 models (Qwen1.5-MoE-A2.7B-Chat, Qwen2.5-3B/7B-Instruct, Qwen3-8B, Qwen3-32B) across all 5 difficulty splits, using validation loss (each variant’s own best hyperparameter configuration, selected exactly as Li et al.’s own criterion) alongside real online-execution pass@1–5. The two loss functions are not comparable by raw validation-loss magnitude: focal loss down-weights easy examples by construction, so it sits on a structurally lower scale than plain cross-entropy’s. We therefore compare validation loss only *within* a loss family, and use real pass@5 (comparable across both) to check empirically whether that choice matters. Within each family, Drop has the lowest validation loss at every model $\times$ difficulty combination tested, in both CE and focal, with zero exceptions. Real pass@5 barely discriminates: averaged across the 4 cross-model-comparable variants over all 5 models and 5 difficulties, CE and focal differ by 0.06pp (41.96% vs. 41.90%, $n=40$ configurations each). Qwen3-8B’s larger 6-variant grid shows a similarly small gap (0.11pp), and the three DM-1-only data treatments (Drop/Crop/Crop-and-Re-verify) differ by under 1pp. Ranking the four comparable variants (Drop/Crop $\times$ CE/focal) by real pass@5 within each of the 25 model $\times$ difficulty combinations and awarding rank-based Borda-count points (1st place scores 3, 2nd scores 2, 3rd scores 1, last scores 0, summed across all 25 combinations) gives Drop-CE 46 points, ahead of Crop-CE (37), Crop-focal (34), and Drop-focal (33), confirming the validation-loss preference under a metric without the CE-vs-focal comparability problem. **We commit to Drop-CE**, uniform across every model and difficulty split, as the recipe used everywhere else in this paper (CE over focal for simplicity and closer fidelity to Li et al.’s own plain-cross-entropy paper). One exception is worth stating: Qwen3-32B shows Crop beating Drop on real pass@5 by a non-trivial margin at several difficulties (e.g. DM-1: Crop-CE 78.4% vs. Drop-CE 75.2%; DM-2: 75.2% vs. 68.8%), despite still losing on validation loss there.

Menu size: consistently small across every split. Table 12 reports how many *distinct* programs appear anywhere across all questions’ top-5 decoded candidate sets, per model, DART-Math difficulty level, and

Table 12: The (F9) small-fixed-menu property holds on every split: the number of *distinct* programs appearing anywhere across all questions’ top-5 decoded candidate sets, Drop-CE recipe, per model and DART-Math difficulty level, on training ($n=1,250/\text{difficulty}$), validation ($n=250/\text{difficulty}$), and test ($n=500/\text{difficulty}$) data.

Model	Split	DM-1	DM-2	DM-3	DM-4	DM-5
Qwen1.5-MoE-A2.7B	train	8	10	9	5	9
	val	8	9	8	5	9
	test	8	9	9	5	9
Qwen2.5-3B	train	12	13	11	13	11
	val	8	10	9	13	11
	test	10	11	10	13	10
Qwen2.5-7B	train	10	11	10	12	20
	val	10	5	7	10	13
	test	10	9	6	11	13
Qwen3-8B	train	13	13	11	10	8
	val	10	10	10	9	8
	test	12	9	9	10	8
Qwen3-32B	train	19	22	16	12	17
	val	19	19	15	11	14
	test	21	21	16	11	15

data split, out of up to 500 possible (one per question), including identity: the menu stays small, at most 22 programs in every cell of the table, and the same small menu appears on the training, validation, and test splits alike for the same model $\times$ difficulty cell ((F9)), so this is a property of the trained router itself, not a test-time or generalization artifact.

B Compute Infrastructure and the Error-Tagging Pipeline

B.1 Compute infrastructure and hardware

Experiments ran on a shared university GPU cluster spanning A100, H100, H200, and B200 nodes. Greedy-decode-based stages (Monte Carlo Tree Search (MCTS), full-dataset evaluation) are pinned to a single GPU architecture per run, since bf16 non-associativity makes greedy decoding diverge slightly across architectures (section A.6). In total, this paper’s reported results used roughly 2,000 GPU-hours (see table 13), plus separate large language model (LLM) usage detailed below. This figure covers search/evaluation compute only and excludes development and debugging run time.

The LLM used for qualitative error tagging runs entirely off this cluster on separate internally hosted infrastructure (one DGX-class node, $8 \times$ A100 40GB) and is not accounted in the GPU-hours. Across its error-categorization use and a separate solver competence check (section B.2), it burned roughly 60M tokens total.

B.2 MiniMax-M2.7-AWQ as LLM-assisted error tagger

The qualitative error analysis in section 4 uses an internally hosted LLM (a locally served, AWQ-4-bit-quantized MiniMax-M2.7 checkpoint, `cyankiwi/MiniMax-M2.7-AWQ-4bit` on Hugging Face, abbreviated MiniMax-M2.7-AWQ here) for one narrow purpose: labeling how close a wrong answer already is to the ground truth, `related` or `unrelated` (*Closeness classification* below). On questions where an edited program’s answer was already established correct by our strict math grader, it is first prompted to describe *why* the base program’s answer was wrong, but that description is scratch reasoning, never itself reported or used elsewhere; only the final `related`/`unrelated` label is. MiniMax-M2.7-AWQ never decides correctness itself, and takes no part in the mechanical error taxonomy below.

Table 13: GPU-hours by what the compute went toward (rounded).

Stage	GPU-hours	Architecture
<i>DART-Math track</i>		
MCTS (5 difficulties $\times$ 5 models)	~ 800	B200
Router training & recipe selection (section A.8), incl. the test-split backfill above	~ 865	A100
<i>MMLU-Pro track</i>		
MCTS (14-domain, Qwen3-8B)	~ 40	A100
Router training & recipe selection	~ 150	A100
<i>Supporting analyses</i>		
Baseline repro., Out-of-Distribution (OOD) transfer & program-of-layers (POLAR)-literal cross-check (section 4.1)	~ 130	H100
Prompt-format sweep (section A.2)	~ 60	H200
Misc. validation (chat-template ablation, MCTS engine checks, MMLU letter-bias)	~ 25	mixed

The tagger is only ever shown the exact text the grader scored (by design, so it cannot critique content that was never causal to the grade), and base models frequently emit the boxed answer *before* any reasoning, so there is usually no genuine derivation left to show once trailing hallucinated continuation is stripped.

Taxonomy. We derive the error taxonomy mechanically, in code, from the same boxed-answer extraction our strict grader uses; MiniMax-M2.7-AWQ plays no role in this step: `\unboxed` (no `\boxed{}` at all), `\malformed boxed` (an empty `\boxed{}`), and `\wrong final answer` (a non-empty boxed answer that simply differs from ground truth), plus a deterministic `\prompt template echo` pre-filter for answers that parrot our own prompt’s unfilled `\boxed{ANSWER}` placeholder (section A.2 covers this failure mode in more depth). The paper’s full-corpus tagging pass (all 5 models, all 5 DART-Math difficulties) resolves to 88.2% `\wrong final answer` overall, with `\unboxed` concentrated almost entirely in one model (Qwen3-32B, 59.3% of its own errors, vs. $\leq 1.1\%$ for every other model); table 14 below gives the full per-model breakdown, and section 4.3 discusses what it means for [F6](#).

Closeness classification. For the `\wrong final answer` category specifically, MiniMax-M2.7-AWQ classifies how close the model’s wrong boxed answer is to the ground-truth answer, shown only the two answers: `\related` if there is any discernible connection, such as close in value, same order of magnitude, a clean scaling or sign relationship (e.g. off by a constant factor, or a sign flip), or the same expression type with a different coefficient. It is `\unrelated` otherwise.

Solver competence check. As a sanity check on whether MiniMax-M2.7-AWQ has any real competence to inform even its descriptions (not proof that they are correct), we separately benchmark it as a *solver* on our own reconstructed DART-Math split (section A.1). Over the full 10,000-question run (all five difficulty levels) under `polar_oneshot` (section A.2), with generation capped at 32,000 tokens since the model runs in thinking mode (our usual 50-token budget, sized for non-reasoning models, would not fit a thinking trace at all), it reaches 96.15% accuracy (99.4% of responses contain a `\boxed{}` at all, 0.05% an empty one, 0.66% a prompt-template-echo placeholder).

Truncation and rescue. Among the models being tagged, Qwen3-32B produces markedly more free-form prose before reaching a `\boxed{}` span than the other four, and is cut off before getting there far more often, the same `\unboxed` mechanism as section A.2. None of these truncated generations contain a literal `<think>` tag. Table 15 reports, for every `\wrong final answer` record, whether MCTS found at least one valid fixing program (rescued), against the closeness label: programs correct a numerically close miss more easily than a structurally different one, in every model (57.9–83.1% vs. 36.9–62.7%).

Table 14: Error-category breakdown for every identity-wrong record, all 5 DART-Math difficulties pooled, per model. Taxonomy and methodology above.

Model	wrong final answer	unboxed	prompt template echo	malformed boxed	Total
Qwen1.5-MoE-A2.7B	7,953 (98.7%)	87 (1.1%)	1 (0.0%)	16 (0.2%)	8,057
Qwen2.5-3B	7,668 (99.0%)	74 (1.0%)	0 (0.0%)	0 (0.0%)	7,742
Qwen2.5-7B	6,299 (100.0%)	0 (0.0%)	0 (0.0%)	3 (0.0%)	6,302
Qwen3-8B	6,599 (99.9%)	0 (0.0%)	2 (0.0%)	4 (0.1%)	6,605
Qwen3-32B	2,548 (39.1%)	3,857 (59.3%)	102 (1.6%)	2 (0.0%)	6,509
All models	31,067 (88.2%)	4,018 (11.4%)	105 (0.3%)	25 (0.1%)	35,215

Table 15: Rescue rate (share of `wrong final answer` records for which MCTS found at least one valid fixing program) by closeness label, all 5 difficulties pooled, per model. Closeness methodology above.

Model	Related n	Related rescued	Unrelated n	Unrelated rescued
Qwen1.5-MoE-A2.7B	6,653	57.9%	1,215	41.3%
Qwen2.5-3B	6,822	64.3%	784	36.9%
Qwen2.5-7B	5,661	66.3%	587	37.6%
Qwen3-8B	5,841	63.5%	700	43.6%
Qwen3-32B	2,355	83.1%	185	62.7%
All models	27,332	64.6%	3,471	41.3%

C Additional MCTS Program-Analysis Results

This section collects the rest of our own MCTS reproduction results that did not fit in section 4, on the same 5 models and 5 DART-Math difficulties. It has two parts: reproductions of several of Li et al. (2026)’s own diagnostic figures, matching what section 4 reports numerically (section C.1), and additional analyses with no equivalent in POLAR’s paper, extending several main-body findings to their full per-model, per-difficulty detail (section C.2).

C.1 Li et al.’s figure reproductions

Table 16 and fig. 10 reproduce Li et al. (2026)’s own skip-and-repeat gain and recurrence-scaling results; their recurrence-vs-skip figure is instead embedded in section 4 as fig. 5, alongside [F3](#). Figure 11 extends this with the full execution-depth-budget breakdown, and table 17 reports the segment-length share behind [F4](#).

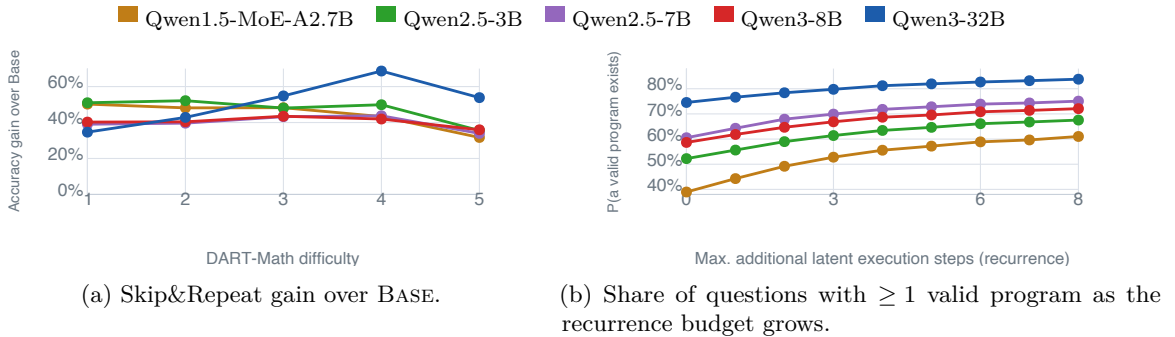


Figure 10: (a) Accuracy gained by allowing skip and repeat together, over the standard forward pass alone, at each DART-Math difficulty level (1 = easiest, 5 = hardest); same Gain column as table 16 above. (b) Share of questions with at least one valid program, as more extra latent execution steps (recurrence) are allowed beyond the standard forward pass depth, pooled over all difficulties, the recurrence-budget half of [F3](#). One line per model in both panels.

Table 16: Combining skip and repeat always beats either alone ([F1](#)): Skip&Repeat’s gain over BASE ranges from +31.7 to +68.7 accuracy points across all 5 models and every DART-Math difficulty. BASE = identity already correct. Skip = identity or a skip-only valid program exists. Repeat = identity or a repeat-only valid program exists. Skip&Repeat = identity or any valid program exists. Gain = Skip&Repeat – BASE.

Model	Difficulty	BASE	Skip	Repeat	Skip&Repeat	Gain
Qwen1.5-MoE-A2.7B	DM-1	22.7%	34.8%	67.1%	73.0%	+50.2
	DM-2	22.8%	35.8%	64.8%	71.0%	+48.2
	DM-3	19.6%	33.5%	61.4%	67.8%	+48.3
	DM-4	20.0%	32.2%	58.4%	63.2%	+43.3
	DM-5	12.2%	19.4%	39.1%	43.9%	+31.7
Qwen2.5-3B	DM-1	31.6%	54.8%	76.5%	82.7%	+51.0
	DM-2	30.1%	53.5%	73.4%	82.2%	+52.2
	DM-3	24.6%	44.5%	66.0%	72.6%	+48.0
	DM-4	18.6%	40.6%	60.5%	68.5%	+49.9
	DM-5	8.0%	21.6%	36.9%	43.5%	+35.5
Qwen2.5-7B	DM-1	50.0%	65.4%	84.9%	89.0%	+39.0
	DM-2	48.6%	66.0%	83.9%	88.2%	+39.6
	DM-3	38.0%	56.0%	76.3%	81.2%	+43.2
	DM-4	31.8%	52.0%	69.3%	75.5%	+43.8
	DM-5	16.7%	29.0%	44.1%	50.6%	+34.0
Qwen3-8B	DM-1	48.6%	67.3%	83.8%	88.8%	+40.2
	DM-2	45.9%	62.8%	81.7%	86.3%	+40.4
	DM-3	34.3%	53.3%	71.9%	77.8%	+43.5
	DM-4	29.7%	48.5%	64.7%	71.7%	+42.0
	DM-5	11.2%	24.9%	41.0%	47.2%	+35.9
Qwen3-32B	DM-1	62.5%	84.4%	94.8%	97.1%	+34.6
	DM-2	53.2%	81.4%	93.4%	96.0%	+42.8
	DM-3	36.7%	71.9%	88.0%	91.5%	+54.8
	DM-4	17.2%	60.1%	80.5%	85.9%	+68.7
	DM-5	5.0%	31.4%	52.3%	58.8%	+53.8

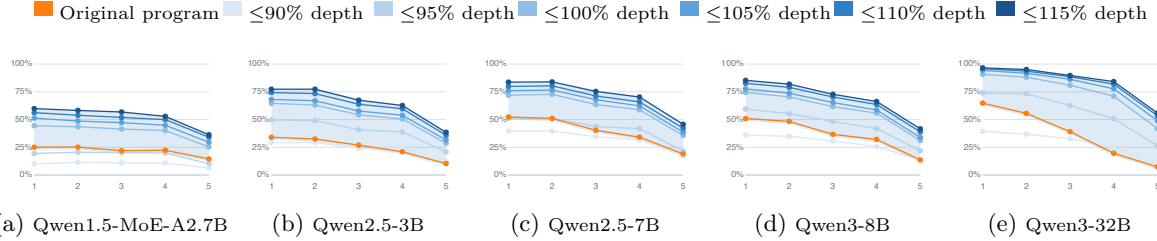


Figure 11: The recurrence-budget evidence behind [F3](#), from a depth-budget angle: best accuracy achievable using only a program (identity or found) whose executed length is at most the given budget, as a percentage of the standard forward pass depth (DART-Math difficulty 1 = easiest to 5 = hardest). We are using the same per-model panel as Li et al. (2026) for our reproduction.

Table 17: Segment-length share of the shortest valid program per solved question, pooled over all 5 DART-Math difficulties, per model: the [F4](#) comparison. POLAR reports 54.5% single-layer and > 66.7% (over two-thirds) at-most-two-layers.

Model	n segments	Single-layer	At-most-two-layers
Qwen1.5-MoE-A2.7B	9,097	46.5%	73.8%
Qwen2.5-3B	12,575	32.6%	58.4%
Qwen2.5-7B	12,410	37.4%	63.1%
Qwen3-8B	13,503	32.7%	58.5%
Qwen3-32B	18,825	21.1%	42.7%
POLAR (reference)	–	54.5%	> 66.7%

C.2 Results with no PoLar equivalent

This subsection collects additional analyses with no equivalent in POLAR’s paper. Figure 12 tracks menu coverage as the candidate menu size K grows; figs. 13 and 14 give the per-model share and length of shorter-than-identity programs, the pooled counterpart to (F2)’s $C \rightarrow C/W \rightarrow C$ split. Figures 15 and 16 give two further results not in the main body: per-model, per-difficulty breakdowns of the same search output that section 4 discusses only at the pooled level, how much of each difficulty is already solved, rescued, or left unsolved, and which operation class (skip-only, repeat-only, or both) the working programs use. Figures 17 and 18 extend fig. 6’s per-layer skip/repeat analysis to the remaining models. The single-segment-reversion robustness check behind (F7) (fig. 9) covers $n = 20,714$ reversions across all 25 model/difficulty combinations, over 19,622 distinct rescuing programs.

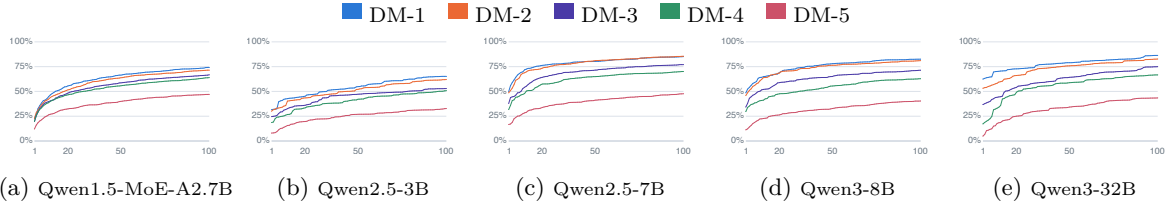


Figure 12: The per-model, per-difficulty view behind (F5): menu generalization degrades with difficulty on every model, coverage still climbing at $K=100$ on the hardest difficulty but nearly flat by $K=30$ on the easiest. Menu coverage of solved questions as the menu size K grows, one line per DART-Math difficulty, one panel per model. K on a log scale.



Figure 13: The per-model, pooled-over-difficulty counterpart to (F2) (Occam’s razor): share of all questions (not split by whether the standard forward pass already answered correctly) admitting a program shorter than it, by DART-Math difficulty. POLAR’s F2 uses the $C \rightarrow C/W \rightarrow C$ split of this measure instead, pooled over difficulty. The prose numbers for that split are in section 4.2 and section 4. Qwen3-8B is omitted here since it already has its own dedicated figure in the main body (fig. 4). One panel per remaining model, all four sharing the leftmost panel’s y-axis (0–100%, same scale throughout).

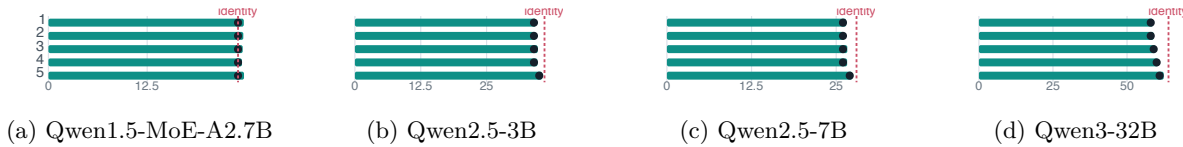


Figure 14: The (F2) split’s program-length counterpart. Same split as fig. 13, but showing the shortest valid program’s actual executed length rather than just whether it beats identity. The dotted line marks the identity (standard forward pass) length D for reference. Qwen3-8B again omitted (fig. 4 in the main body). One panel per remaining model, by DART-Math difficulty, all four sharing the leftmost panel’s y-axis.

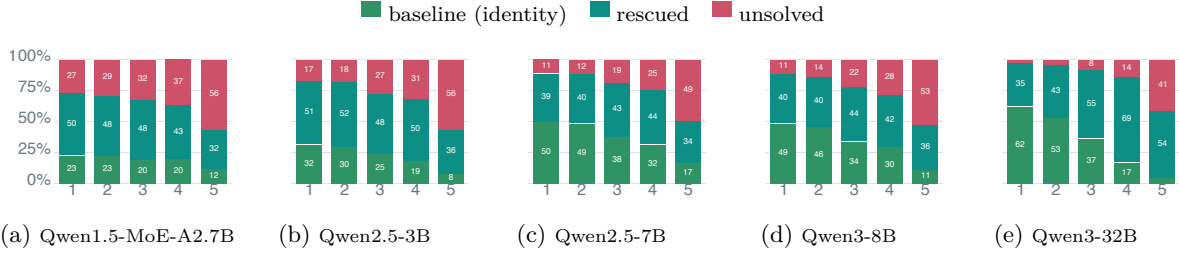


Figure 15: The baseline/rescued/unsolved split behind (F5)’s menu coverage. Of every question: the share already answered correctly by the standard forward pass (baseline); the share the standard forward pass got wrong but for which the search found at least one valid program (rescued); the share for which nothing valid was found (unsolved). One panel per model, by DART-Math difficulty, all five sharing the leftmost panel’s y-axis.

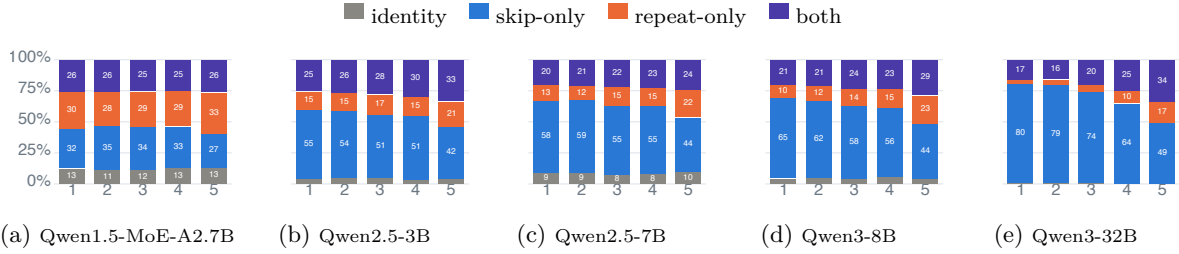


Figure 16: The (F1) solvability split by operation class. Of every solved question, whether the standard forward pass already works (identity), a program using only skips works, a program using only repeats works, and a program using both works (shortest-valid-program lens). These four checks are independent, so a single question can count toward more than one bar and the four bars do not need to sum to 100%. One panel per model, by DART-Math difficulty, all five sharing the leftmost panel’s y-axis.

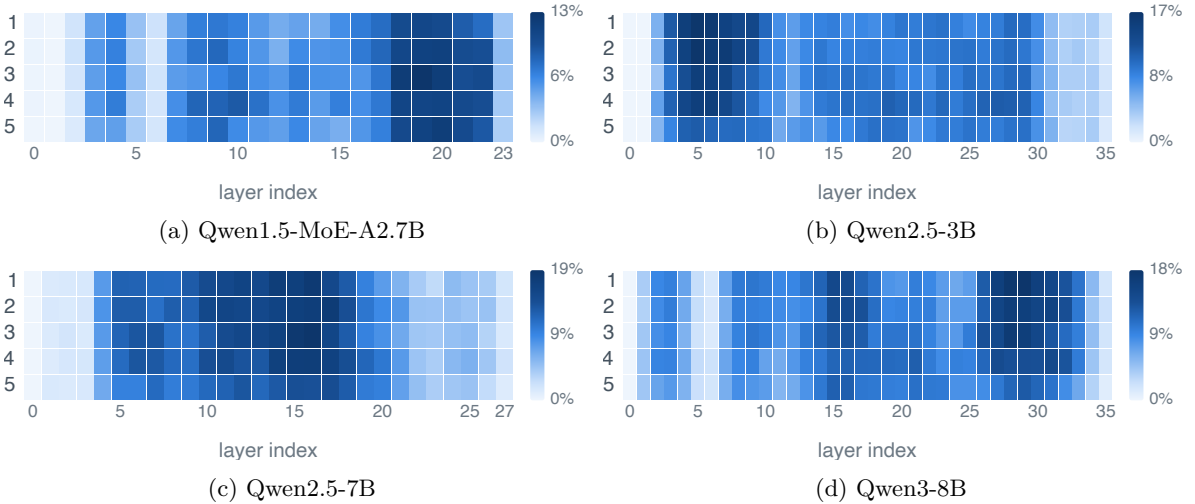


Figure 17: The per-model layer-position evidence behind (F3): the earliest layers are almost never skipped in any model, consistent with the early load-bearing region we find independently in fig. 2. This panel shows the share of shortest-valid programs that skip each layer, by layer index and DART-Math difficulty. Qwen3-32B is omitted here since it already has its own dedicated figure in the main body (fig. 6).

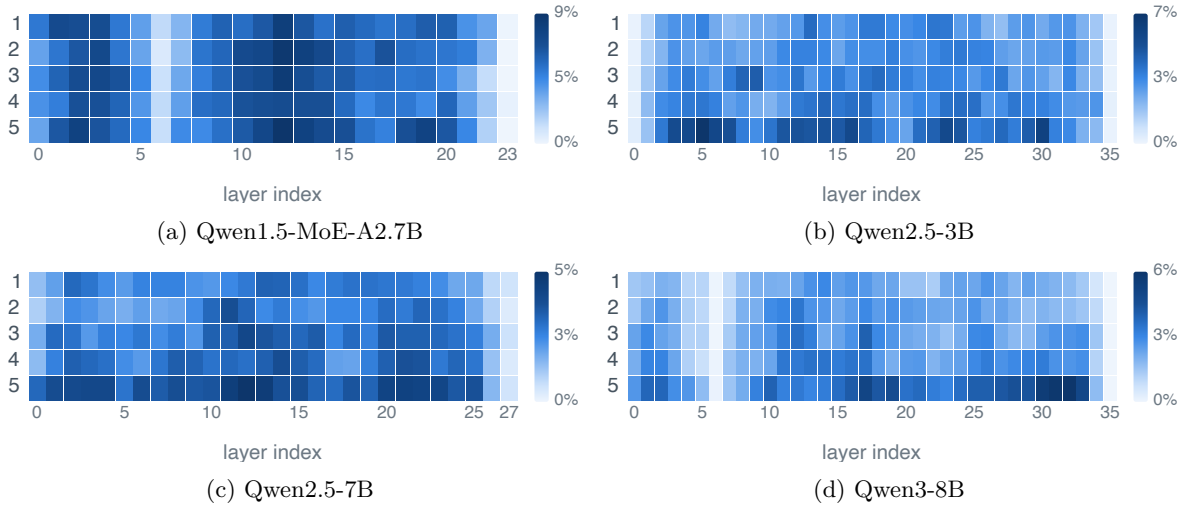


Figure 18: The repeat-side counterpart to fig. 17, also part of the [F3](#) layer-position evidence. Share of shortest-valid programs that repeat each layer, by layer index and DART-Math difficulty. Qwen3-32B again omitted (fig. 6 in the main body).

D Program-of-Layers on MMLU-Pro

MMLU-Pro’s own answer keys (Wang et al., 2024) carry no real letter bias. Conditioned on `num_options`, the dominant group in our split (4,220 of 5,083 train rows, 83%, all with `num_options` = 10) is close to uniform (per-letter frequency 9.4–11.3% against a 10.0% random baseline). We find no discrete-letter bias for any `num_options` bucket we checked (3–10). So any bias we observe below is a property of the model or its Monte Carlo Tree Search (MCTS)-discovered programs, not the data. The rest of this section gives the full evidence that many MCTS-discovered programs on this benchmark answer correctly for a specific option arrangement rather than a specific question, and collapse when that arrangement changes (sections D.1 and D.2).

D.1 Option-order robustness: MCTS-discovered programs track position, not content

A program that has genuinely learned to answer a question should still get it right if the question’s options are presented in a different order. A program that has instead keyed on which letter or position the correct answer occupied should not. We test this directly: re-score an MCTS-discovered program after randomly reshuffling its query’s option order, leaving question content untouched, and compare against the program’s original-order accuracy. This is one of our two main MCTS-on-MMLU-Pro findings: many MCTS-discovered programs on this benchmark are unable to survive an option reshuffle, meaning they have effectively overfit to the correct letter rather than solved the question.

We repeat the option-reshuffle check on Qwen3-8B’s full search (all 12,032 MMLU-Pro queries, no train/val/test subsetting), to confirm the finding below is not an artifact of subsetting to train/val/test splits.

Identity is close to shuffle-invariant; rescuing programs are not. As a control, we reshuffle option order for the *entire* evaluation set (not just rescuing queries) and re-score under the unmodified identity program: accuracy is unchanged within noise, 42.2% → 41.9% ($n = 12,032$), even though 29.9% of individually correct answers flip to wrong and 21.3% of individually wrong answers flip to correct under the identical reshuffle – net-flat accuracy masking substantial per-query churn, consistent with ordinary decision noise rather than systematic position exploitation. Programs discovered by MCTS specifically to rescue an identity failure behave completely differently under the same manipulation: on the rescuing population ($n = 6,866$, identity wrong, program correct, under the original arrangement), program accuracy collapses from 94.1% to 17.7%, a 76.4 percentage-point drop, not far above the uniform-random floor (roughly 10% for this dataset’s dominant 10-option questions, section D).

The fragility is not exclusive to rescuing programs, but rescuing programs are markedly worse. Repeating the check on every query with any valid non-identity program ($n = 11,938$, 99.2% of the dataset, almost the whole benchmark) and splitting by whether identity itself was already correct: programs found where identity already succeeds are also reshuffle-fragile ($96.8\% \rightarrow 36.6\%$, -60.2pp) but recover well above the random-guess floor, while rescuing programs collapse further still, matching the rescuing-only measurement above ($94.1\% \rightarrow 17.7\%$). Reshuffle-fragility is therefore a general property of MCTS-discovered non-identity programs here, not an artifact of the rescue mechanism specifically, though programs built to correct an identity failure lean on the original option arrangement hardest. In short: the base model is close to shuffle-invariant, but every population of MCTS-discovered non-identity programs we measure tracks the arrangement a question happened to be presented in, not what it is actually asking.

D.2 Router reproduction on MMLU-Pro: a random-valid-program control

Section 4.4 documents the router’s pass@1 identity-collapse on DART-Math and yet a genuine pass@5 rescue signal underneath it. We separately trained and evaluated a router directly on MMLU-Pro, on the 5,083 train rows introduced above and a 2,800-question held-out test split, to test whether the position-overfitting result in section D.1 predicts a parallel router-level failure: if what MCTS labels as “correct” on this benchmark is disproportionately a position artifact rather than a stable function of question content, a router trained on that supervision has little content-based signal left to generalize from, and should not be expected to rank real candidates any better than chance.

Pass@1 collapses onto identity, exactly as on DART-Math. Across every recipe we swept (down-weighting, class-balanced focal loss, length-preference reweighting, Drop/Crop repeat-segment parsing, section A.8), the router’s top-1 prediction is the identity program for every one of the 2,800 held-out test queries, with identity accuracy 43.9%. This matches section 4.4’s DART-Math finding and confirms it is not an artifact specific to that dataset’s answer format.

A pass@5 margin over identity survives, but so does an equally large margin for random guessing. Pass@5 for Drop-CE (section A.8’s DART-Math-winning recipe) reaches 62.6%, above the 43.9% identity floor (the rest of our recipe sweep lands lower, 53–57%; section A.8 covers those variants). But on a forced-choice benchmark with as few as 3–10 options, trying several candidates and taking the best can look like a real gain with zero learned signal, so we build a control: 5 shared valid programs sampled uniformly from the router’s own action grammar, independent of training, graded identically, repeated over 5 seeds. This control averages $45.3\% \pm 8.8\%$ at pass@5 ($17.0\% \pm 9.7\%$ at pass@1); only Drop-CE clears it, by roughly two standard deviations. That margin, though, does not behave like a learned one: retraining Drop-CE’s winning configuration 5 independent times (weight-init/data-order seed only) reproduces *exactly* the same pass@5 figure (62.6% to full floating-point precision) every time, because each checkpoint decodes an identical top-5 candidate set, question by question, across all 2,800 test queries (table 18) – despite the 5 checkpoints having substantially different weights, validation loss (1.0920–1.0953), and pre-decode logits. If the router’s non-identity candidates tracked each question’s content, different initializations should disagree at least occasionally on which candidates to propose; zero disagreement across five runs with genuinely different weights instead points to training converging on a fixed, low-content-dependence attractor, not evidence the router has learned to read questions. Our working hypothesis is that this fixed attractor is itself a small menu of programs chosen to jointly cover the answer distribution well, beating 5 randomly sampled programs the same way a fixed, well-chosen menu beats a random one elsewhere in this study (F5), rather than a per-question, content-aware prediction.

Read against section D.1, this closes the loop the option-order finding opens: MCTS supervision on MMLU-Pro is dominated by a position artifact rather than genuine content signal, and a router trained on it does not recover that signal either. The pass@5 margin it shows is real and reproducible, but structurally fixed rather than per-question: the router-level counterpart of the same failure. This stands in direct contrast to DART-Math, where section 4.4 finds real, MCTS-verified, per-question candidate diversity underneath the same pass@1 collapse. The two benchmarks fail the router in different ways, and MMLU-Pro’s failure mode is the one the reshuffle result predicts: a multiple-choice format gives MCTS-discovered programs a shortcut, the answer’s position, that they can overfit to instead of the question’s content.

Table 18: The Drop-CE router beats a random-valid-program control at pass@5 (62.6% vs. $45.3\% \pm 8.8\%$), but identically across all 5 independently seeded retrains, a structurally fixed margin, not per-question adaptation. Qwen3-8B, 2,800-question held-out test split, same online-execution grading protocol both rows. “Random baseline” draws 5 shared valid programs from the router’s action grammar, independent of training, repeated over 5 seeds (mean $\pm$ std). Drop-CE router retrains that recipe’s winning hyperparameter configuration 5 independent times (weight-init/data-order seed only); all 5 runs give the identical figure.

	pass@1	pass@2	pass@3	pass@4	pass@5
Identity	43.9%	–	–	–	–
Random baseline	$17.0\% \pm 9.7\%$	$25.9\% \pm 8.8\%$	$36.1\% \pm 6.4\%$	$40.4\% \pm 7.8\%$	$45.3\% \pm 8.8\%$
ROUTER (Drop-CE)	43.9%	–	–	–	62.6% (identical, all 5)

E Further Brain-Inspired Machine Learning Approaches

Modular specialization and global coordination. Sparse mixture-of-experts (MoE) architectures introduce conditional computation by routing each token representation to only a subset of feed-forward experts within a layer: selective, but locally scoped, since it does not alter the model’s overall layer-level execution path. Mixture of Cognitive Reasoners (MiCRo) extends the idea of specialization by organizing expert modules around distinct cognitive networks (AlKhamissi et al., 2025). Programs-of-layers instead condition on the whole execution path, which layers run at all and how many times, rather than which experts fire within a fixed layer sequence. Goyal et al. (2021) propose a complementary piece: a shared, bandwidth-limited communication channel through which otherwise-specialized neural modules coordinate, inspired by global workspace theory. We invoke this same idea in section 5: a program-of-layers’ path-level coordination, deciding a sequence of engagements over the whole forward pass, is closer in spirit to this global channel than to MoE’s per-token, per-layer routing. MoE routing stays local and never sees the full path at once.

Brain mechanisms as design inspiration. Hawkins et al. (2019) argue that grid-cell-like location representations, neurons that encode spatial position, run throughout the neocortex and that individual cortical columns learn complete models of objects by combining external sensory input with this location signal. Their picture of many largely independent columns, each building its own model rather than passing everything through one fixed hierarchy, is consistent with program skipping or repeating layers per input (section 4). Unlike (Li et al., 2026), Hawkins et al. invite building computationally based cortical columns rather than routing inputs over existing LLM architectures. Darlow et al. (2025) build neural synchronization and neuron-level temporal processing directly into the architecture (the Continuous Thought Machine), and show it supports adaptive computation, allocating more processing to harder inputs. This is a fundamental biology-based architecture implementation that differs from Li et al., since Li et al. reuse a standard transformer’s existing layers and change only *when* and *how often* they fire, rather than changing what a unit computes internally. Adeel & Bilal (2026) link biophysical models of pyramidal neurons (a common type of cortical neuron) to modulation loops among attention components, and report faster learning at roughly linear complexity on a standard vision-transformer benchmark. Like the Continuous Thought Machine, this operates at a finer grain than Li et al. layer-level programs, but it is further evidence that attention specifically, not just depth or routing, is an active target for brain-inspired redesign.

Measuring the analogy, and borrowing brain subsystems rather than whole architectures. Shah & Khosla (2025) introduce Multi-Level Optimal Transport (MOT), a framework that jointly aligns layers and individual units across networks of different depth, and apply it to align model layers not only against each other but directly against human visual-cortex recordings. Unlike our own analogy in section 5, which stays at the level of a coordinating role rather than a specific mapping, MOT gives a concrete recipe for actually testing whether the layers a program-of-layers revisits align with specific cortical regions, an empirical test we only gesture at here, not one we run. Tran et al. (2025) use top-down modulation, in the style of predictive coding, a theory that the brain constantly predicts its own input and corrects for the difference, to consolidate representations across tasks from very little labeled data, motivated by cortical feedback circuits. The connection to Li et al.’s work is looser here, and we include it as a further example of the

same general pattern: cortical feedback as a design principle for an machine learning (ML) system, rather than as a direct precedent. Gutiérrez et al. (2024) structure a retrieval-augmented large language model (LLM)’s long-term memory after the hippocampal-neocortical memory system with their method, HippoRAG, improving multi-hop retrieval accuracy and speed over iterative retrieval baselines. Memory is orthogonal to the execution-path question this paper asks, but the move is the same: borrow a specific, well-characterized brain system’s computational structure, not just its name, as a design template. Berges et al. (2024) scale a complementary, non-brain-inspired mechanism, their *memory layers* (sparse trainable key-value memory), to large models. We include their work as a contrast case: it treats memory as a separate subsystem too, but without any neuroscience motivation, showing the same architectural idea can arise independently of brain inspiration.

F Communication with the Original Authors

Reproducing Li et al. (2026) was difficult throughout. The released repository¹ covers only router training and evaluation given precomputed Monte Carlo Tree Search (MCTS) programs for supervision, which is itself not released. Three things therefore had to be reconstructed from underspecified prose rather than checked against a reference implementation: the search that produces that supervision (described only as a seven-line pseudocode box), the DM-1–DM-5 difficulty split every reported table is organized around, and the code that produced the paper’s own baseline tables (sections A.1, A.6 and 4.1).

Our attempts to contact Li et al. to inquire about the unpublished MCTS components required to reproduce their work were unsuccessful. Correspondence with the original authors could have resolved many of the reconstruction challenges quickly; instead, we undertook an independent reconstruction and validation effort, documented throughout this paper’s appendix.

G Use of Large Language Models

The authors used general-purpose large language model (LLM) coding assistants (Claude Code) during development: writing and debugging experiment code in the companion repository, and drafting/editing prose and L^AT_EX in this paper. All generated code and text were reviewed and edited by the authors before use; all reported numbers, claims, and conclusions are the authors’ own and were verified against the underlying experiment outputs.

¹<https://github.com/tianyi-lab/PoLar>